| 队伍编号 | MC25002366 |
| --- | --- |
| 题号 | D |

# 基于约束分段预测模型下的货量运输优化调度问题


## 摘要

本文基于历史 15 天各条运输线路的包裹运输量，对未来一天的各条运输线路的包裹运输量的预测和多目标约束的运输调度优化问题，分别进行合适的建模和求解。

**针对问题一**，本文先对历史数据以 10 分钟颗粒度进行可视化分析，发现数据呈现时间分段和周期性波动趋势，十分适合用时间序列 LSTM 预测模型进行预测，为了进一步降低模型误差并提升预测精确度，本文引入了分段拟合的思想，对 10 分钟颗粒度的历史数据，再按照时间段划分为早、午班，将每个班每日的所有数据定为一个周期，再设定训练集和测试集，并对每个周期内的数据划分出高峰段和低谷段，分段进行 LSTM 预测拟合，并引入连续性约束条件使得各个片段之间的预测数据平滑连接。最后对比预测结果与测试集之间的误差，发现有约束的分段预测 LSTM 模型精确度明显较高，并用该模型预测未来一日的货量数据，使得模型具有良好的精确度和适应性。

**针对问题二**，为了获得最优调度结果，本文先将多目标加权转换为单目标优化问题，引入“串点”方法合并包裹量少的路线，根据题目条件设立对应的约束条件，最后采用混合整数线性规划 MILP 求解问题。成功实现了自有车的周转率尽可能高、所有车辆均包裹尽可能高以及总成本尽可能低的三个目标优化，体现了该调度模型很强的泛化能力和现实意义。

**针对问题三**，引入了标准容器变量，本文先用货量拆分规则关联标准容器变量，再结合贪心算法与回溯法来动态调整容器的使用策略。不仅优化了线路分配，还提高了自有周转率、串点效率、单车装载率，并且有效减少了外有车辆的运用数量，从而降低了整体运输成本。

**针对问题四**，为了评估货量预测结果偏差对调度模型优化结果的影响，本文对调度模型进行鲁棒性分析和灵敏度分析，通过对比不同预测误差情况下关键指标的变化幅度，引入动态调整机制来让调度模型在一定范围的预测误差波动时，提高关键指标的稳定性，有效降低了因预测误差带来的成本超支和调度方案失效概率。

综上所述，本文建立了包裹运输量的时间预测模型和运输调度的优化问题，不仅提升了运输效率，还降低了运输成本，具有较强的实践意义和应用价值。



**关键词：** **LSTM　有约束的分段预测模型　贪心算法　灵敏度分析　动态调整**

# 目录


一、 问题重述 …… 1

1.1 问题背景： …… 1

1.2 问题要求 …… 1

二、 问题分析 …… 1

2.1 对问题的整体分析 …… 1

2.2 对问题的具体分析 …… 2

2.2.1 问题一分析 …… 2

2.2.2 问题二分析 …… 3

2.2.3 问题三分析 …… 3

2.2.4 问题四分析 …… 3

三、 模型假设 …… 3

四、 符号说明 …… 4

五、 模型建立与求解 …… 4

5.1 问题一、货量预测模型 …… 4

5.1.1 数据预处理与可视化 …… 4

5.1.2 货量预测 LSTM 模型的建立与求解 …… 6

5.1.3 分段预测模型的建立与求解 …… 11

5.1.4 有约束的分段预测模型的建立与求解 …… 11

5.2 问题二、优化调度决策模型 …… 14

5.2.1 决策变量 …… 14

5.2.2 目标函数 …… 14

5.2.3 约束条件 …… 15

5.2.4 混合整数线性规划 MILP 模型的求解 …… 16

5.3 问题三的模型的建立与求解 …… 18

5.3.1 模型调整 …… 18

5.3.2 模型求解 …… 18

5.3.3 求解结果 …… 19

5.4 问题四的模型的建立和求解 …… 21

5.4.1 问题背景与模型准备 …… 21

5.4.2 模型建立与求解 …… 23
5.4.3 模型优化 …… 24
六、 模型的评价 …… 26
6.1 模型的优点 …… 26
6.2 模型的缺点 …… 26
七、 模型的改进与推广 …… 27
参考文献 …… 28
A 附录 文件列表 …… 28
B 附录 代码 …… 28

# 一、 问题重述

## 1.1 问题背景：

短途运输是物流网络的末端环节，其距离短，资源可重复利用，要求保障时效，任务是把货物从分拣中心配送至营业部。随着电商经济的蓬勃发展，物流需求日益增长，该环节需要众多的运力资源，且用户对包裹的时效要求越来越高，物流运输面临着巨大挑战，需要提前预测短途运输的货量来保障履约质量、控制运输成本、合理利用运力资源。

在短途运输中，车辆周转率直接影响单位运输成本，自有车周转率是自有车承运总需求数与车队自有车辆的比值，提高自有车辆周转率可降低单次运输成本 [1]。其次，提高车辆均包裹数，可提高单辆车的利用率，减少成本 [2]。因此，提高自有车车辆周转率、提高车辆均包裹数、控制运输成本对提高整体运营效率尤为重要。

## 1.2 问题要求

**问题一：** 利用历史货量数据建立货量预测模型，先预测未来 1 天各条线路的货量，再将每条线路的总货量拆解到 10 分钟颗粒度。

**问题二：** 基于问题一的预测结果，考虑三个目标条件：让自有车的周转率尽可能高，让所有车辆均包裹尽可能高、总成本尽可能低。增添每个需求的发运时间不能晚于线路的发运节点，串点涉及的线路不允许超过 3 个等约束条件。要求对该多目标优化问题建立规划模型来求得最优路线规划方案。

**问题三：** 分析各运输需求是否使用一种标准容器，该容器能将装车及卸车时长缩短至 10 分钟，但装载包裹量下降至 800 个，虽然提升了自有车辆利用率，但也降低了车的装载量。该问题基于结合问题一的预测结果和问题二得出的运输需求，重新优化分析各运输需求是否需要这种标准容器。

**问题四：** 现实中，货量预测会因为多种原因出现偏差，该问题需要考虑当问题一的货量预测结果的出现误差时，分析误差对调度模型优化结果的影响。

# 二、 问题分析

## 2.1 对问题的整体分析

首先要对问题一建立货量预测模型，包含数据预处理、LSTM 神经网络和分段预测模型结合，生成精准预测；随后基于问题一的预测结果，对问题二构建多目标 MILP 优化模型，通过加权目标函数和约束条件求解得到最优运输调度方案；问题三先通过货量拆分规则关联标准容器，再结合贪心算法与回溯法动态调整是否使用容器来形成容器使用策略；最终在问题四进行鲁棒性验证，通过模拟货量预测误差变动，考虑对关键指

标的影响，本文构建鲁棒优化模型和灵敏度分析，探测货量预测误差变动对各个关键指标的影响强弱，再引入引入动态调整机制来提升调度方案的鲁棒性，形成抗扰动调度模型，使得模型在一定误差波动范围内的稳定性能更高。

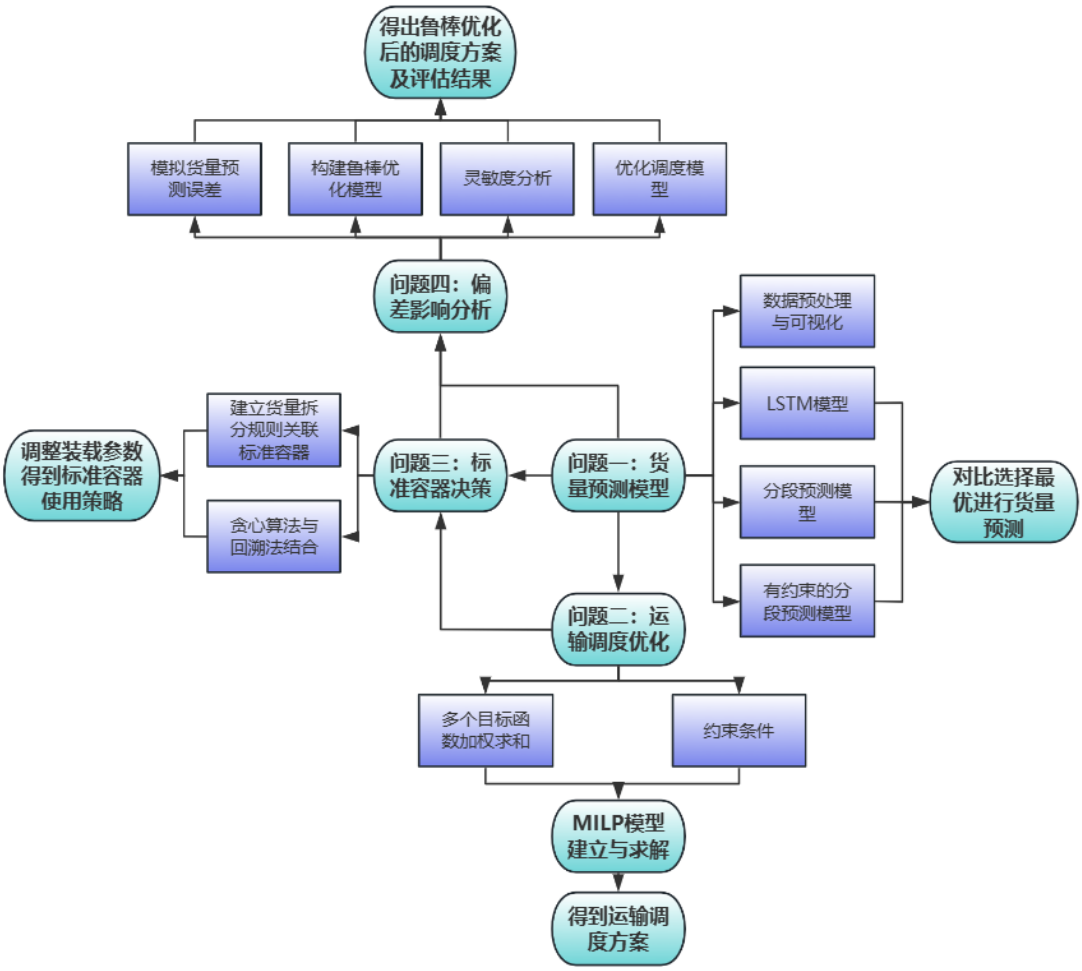



**图 1　问题的整体思路**

## 2.2 对问题的具体分析

### 2.2.1 问题一分析

问题一要求建立货量预测模型，先对未来 1 天各条线路的货量进行预测，再将各线路的总货量拆解到 10 分钟颗粒度后重新进行预测，来为后续的运输需求确定和车辆调度提供依据。从可视化图表中得知，由于包裹量数据具有周期性和季节性波动，因此建立 LSTM 时间序列预测模型是最为合适的选择。为了进一步降低预测误差，提高精确度，本文先将历史数据集以周期划分为各个片段，重新进行预测后再对各个片段添加连续性约束条件，对比这三种方法结果，选出误差最小的预测模型来对未来 1 天的包裹量进行预测。

### 2.2.2 问题二分析

问题二基于问题一预测的 140 条路线的货量数据，将原本的自有车的周转率、总成本、车辆均包裹三个目标的多目标优化问题进行加权求和，转化为单目标优化问题，再根据题目条件建立对应的线性约束条件，建立混合整数线性规划 MILP 模型求解。

### 2.2.3 问题三分析

问题三基于问题一的预测结果和问题二的调度结果，引入标准容器，在优化目标不变的情况下，改变了包裹装载量和装卸时间。对此，本文引入了贪心算法和回溯结合的方法来衡量标准容器对最优解的影响。

### 2.2.4 问题四分析

问题四探讨当货量预测结果出现误差时，对调度模型结果的影响。对此，本文考虑灵敏度分析和对模型进行鲁棒性检验，并引入动态调整机制来提升调度方案的鲁棒性。

## 三、 模型假设

为简化问题，本文做出以下假设：

- 假设 1：包裹量不会在运输时退货。
- 假设 2：串点涉及的线路不能超过 3 个。
- 假设 3：所有车辆载重能力和装车、卸车的时间固定。
- 假设 4：运货的在途时长固定，不受交通阻塞的影响。
- 假设 5：除可串点的情况外，各条线路的运输需求是相互独立的。
- 假设 6：优先使用自有车辆，只有再自有车辆空缺时才调用外部承运商，并不考虑车辆的维护时间。
- 假设 7：附件 2 中历史 15 天的数据具有代表性，能充分反映未来的货量趋势，不受外部因素的影响。

# 四、 符号说明

| 符号 | 说明 |
|---|---|
| $W$ | 对应门的权重矩阵 |
| $\lambda_i$ | 第 $i$ 个目标的权重系数 |
| $S_j$ | 任务 $j$ 合并的线路集合 |
| $J_{merge}$ | 所有串点任务的集合 |
| $F$ | 附件 5 中的车队集合 |
| $V_f$ | 车队 $f$ 的自有车辆集合 |
| $C_l$ | 线路 $l$ 的自有变动成本 |
| $T_j$ | 任务 $j$ 对应的线路 $l$ 的单程在途时长 |
| $S$ | 附件 4 中的可串点集合 |
| $P$ | 车辆均包裹数 |

# 五、 模型建立与求解

## 5.1 问题一、货量预测模型

### 5.1.1 数据预处理与可视化

先对附件所给的数据进行整合与可视化检验，绘制部分线路的包裹量时间序列图，检查数据是否存在季节性模式、周期性波动。以“场地 3 - 站点 83 –1400”和以“场地 3 - 站点 83 –0600”11.30-12.15 号的数据为例，绘制可视化图表：

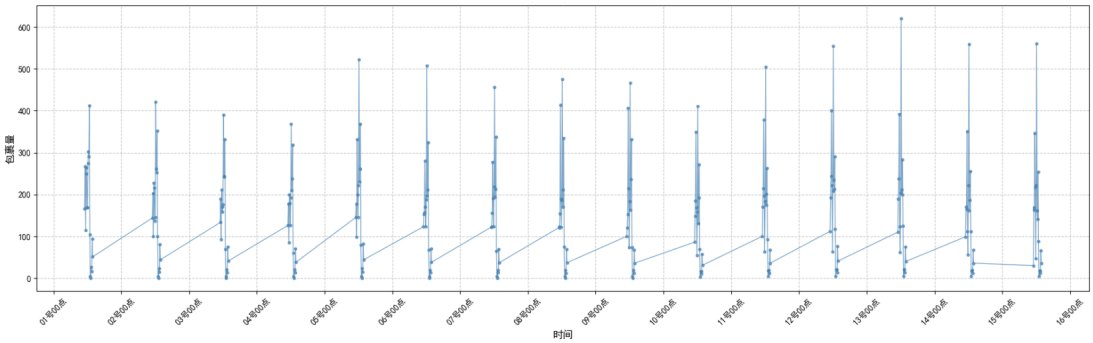



**图 2 “场地 3 - 站点 83 –1400”包裹量时间序列图（散点形式）**

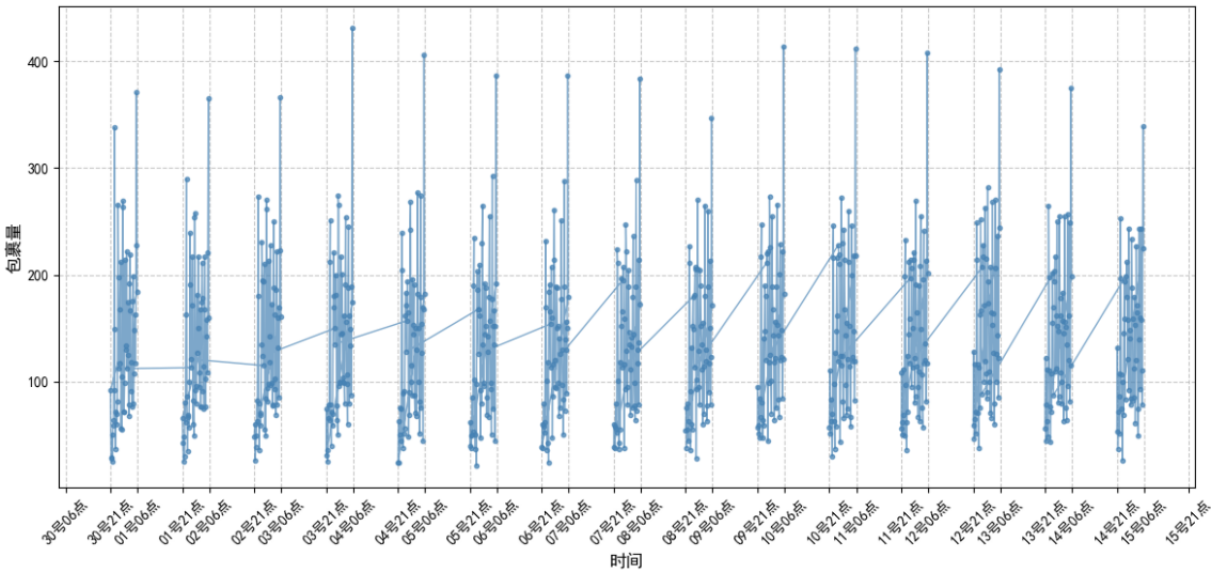



**图 3　“场地 3 - 站点 83 –0600”包裹量时间序列图（散点形式）**

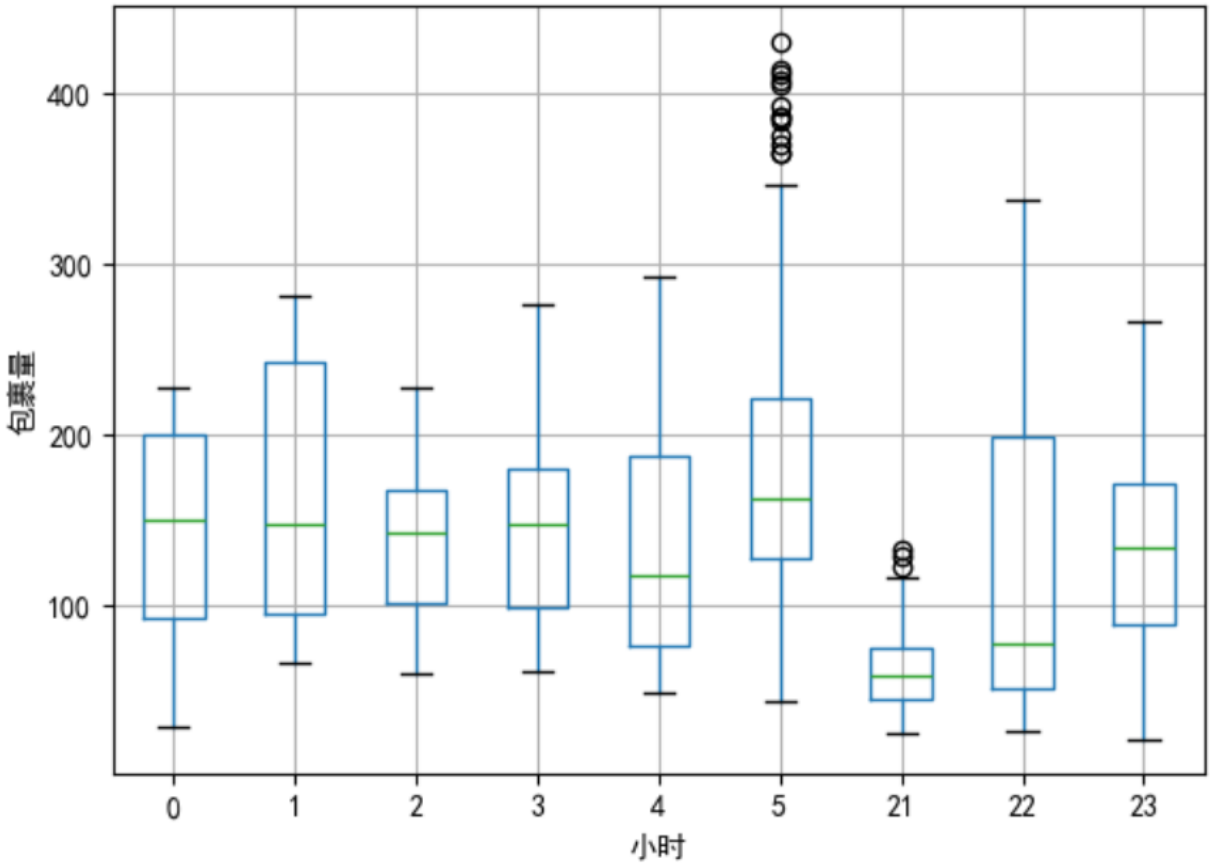



**图 4　每小时包裹量分布图**

从可视化图表可以明显看出场地 3 - 站点 83 –0600 的包裹量整体呈现出一定的波动性和周期性，在每天的 6 点附近出现最高峰。并且每天的 6-11 点与 14-21 点均无包裹量，符合题目假设的条件。

### 5.1.2 货量预测 LSTM 模型的建立与求解

在本问题中，物流货量的时序特性复杂，包裹量变化呈现出与时间相关的波动性和周期性，货量预知数据存在线路偏差、未下单货量、订单取消的不确定性，而且特征维度多，需要基于历史货量数据和预知货量等特征类型预测货量。而 LSTM 是一种适用于时间序列预测的深度学习模型，能处理复杂时序数据的长期依赖关系，可捕捉周期性特征，支持多变量输入，因而是用来建立本问题货量预测模型的最优方法之一。针对本问题，LSTM 的目标是根据已有的历史货量数据和预知数据，对未来 1 天各条线路的货量进行预测。定义每条线路 $l$ 的历史货量数据为 $\{Q_l(t_1), Q_l(t_2), ..., Q_l(t_n)\}$，其中 $Q_l(t_i)$ 是线路 $l$ 在时间点 $t_i$ 的货量，$n$ 是时间序列的长度。

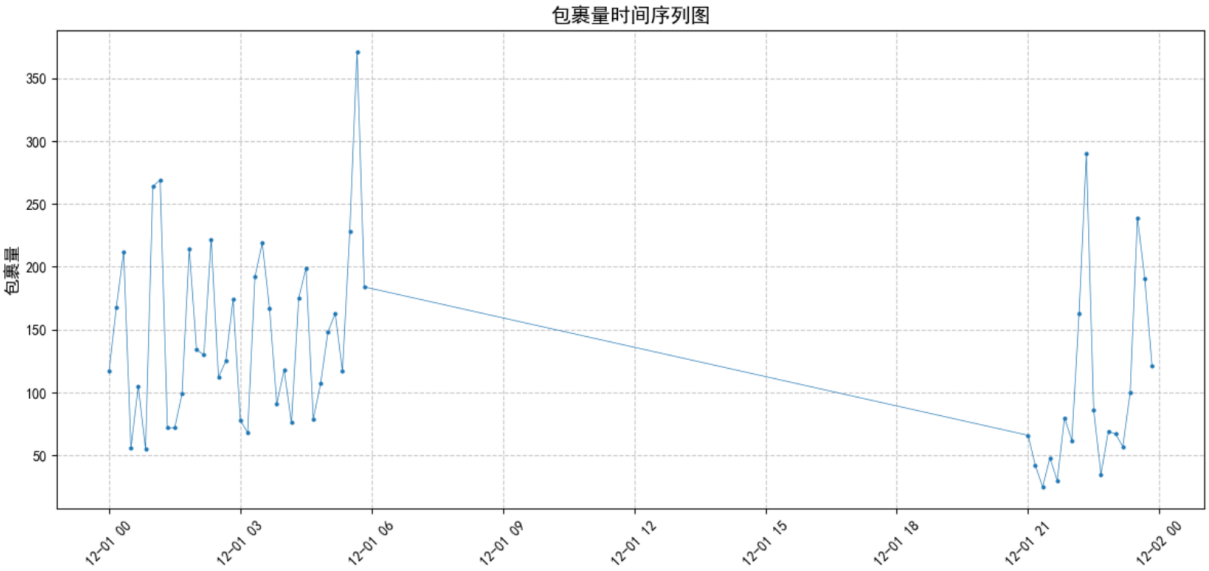



**图 5　场地 3 - 站点 83 - 0600 一日包裹量趋势**

从一日包裹量趋势图可以看出，包裹量可以按照发运节点规律划分为早、午班数据片段。早班周期是当日的 21 点到次日的 6 点，一共是 9 小时即 54 个 10 分钟点，午班周期是 11 点到 14 点，一共是 3 个小时即 18 个 10 分钟点，然后将两个周期合并成周期矩阵，数学表达式如下：

$$\text{早班周期：}\ X_l^{\text{morn}} = \{Q_l(t) | t \in [21:00, 6:00)\} \tag{1}$$

$$\text{午班周期：}\ X_l^{\text{noon}} = \{Q_l(t) | t \in [11:00, 14:00)\} \tag{2}$$

$$\text{周期矩阵：}\ \mathbf{X}_l^{\text{cycle}} = [X_l^{\text{morn}} \| X_l^{\text{noon}}] \tag{3}$$

LSTM 的核心是根据时间序列数据 $X_l^{total} = \left[\sum_{\tau=1}^{54} Q_l^{\text{morn}}(\tau), \sum_{\tau=1}^{18} Q_l^{\text{noon}}(\tau)\right]_{1\times 15}$ 预测未来一天的货量，$\sum_{\tau=1}^{54} Q_l^{\text{morn}}(\tau)$ 是第 τ 天早班周期的总货量，$\sum_{\tau=1}^{18} Q_l^{\text{noon}}(\tau)$ 是第 τ 天午班周期的总货量。LSTM 的输出变量为预测货量 $\hat{Q}_l^{\text{day}}$，数学表达式如下：

$$\hat{Q}_l^{\text{day}} = \text{LSTM}(X_l^{total}, \theta) \tag{4}$$

其中，$X_l^{total}$ 是日总货量特征矩阵即输入的历史数据，$\theta$ 是 LSTM 的参数，$\hat{Q}_i^{\text{day}}$ 是线路 l 未来 1 天的总货量预测值。

在操作中，用 LSTM 来训练已有的历史货量数据, 并利用训练得到的模型来预测未来一天内每条线路的货量。**LSTM 的基本结构**可以通过以下数学公式表示:

1. 遗忘门:

$$f_t = \sigma(W_f \cdot [h_{t-1}, X_l^{total}] + b_f) \tag{5}$$

过滤历史无关信息（如异常波动）。

2. 输入门:

$$i_t = \sigma(W_i \cdot [h_{t-1}, X_l^{total}] + b_i) \tag{6}$$

其中，$X_l^{total}$ 表示当前输入向量 (即历史货量数据或外部特征如节假日、天气、促销活动等)，$W_f$ 是权重矩阵，$b_f$ 是偏置项，$\sigma$ 是 sigmoid 激活函数。

3. 候选记忆单元:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, X_l^{total}] + b_C) \tag{7}$$

生成当前时刻新的候选记忆单元状态。

4. 细胞状态更新:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{8}$$

更新当前时刻的单元状态，是通过遗忘历史信息与引入新信息的加权合成，结合遗忘门与输入门更新长期记忆

5. 输出门:

$$o_t = \sigma(W_o \cdot [h_{t-1}, X_l^{total}] + b_o) \tag{9}$$

决定当前隐藏状态的输出信息。

6. 隐藏状态:

$$h_t = o_t \odot \tanh(C_t) \tag{10}$$

最后输出的隐藏状态代表了当前时刻的记忆信息。

**模型架构设计：**针对物流货量预测任务，构建双层堆叠 LSTM 网络，其中第一层 LSTM 负责返回完整序列供下一层使用，第二层 LSTM 仅返回最终输出，全连接层输出预测值。

**构造监督学习数据：**将时间序列转换为监督学习样本，定义时间窗口参数:

- $n_{\text{steps_in}} = 14$：用过去 14 天数据预测第 15 天数据。
- $n_{\text{steps_out}} = 1$：单步预测

将数据转换为函数:

$$X = [Q_{t-6}, Q_{t-5}, \ldots, Q_t] \square y = Q_{t+1} \tag{11}$$

**Min-Max 归一化处理：**以达到消除量纲差异，加速模型收敛的作用，并避免大数值特征主导训练过程。

$$Q^{\text{scaled}} = \frac{Q - Q_{\text{min}}}{Q_{\text{max}} - Q_{\text{min}}} \tag{12}$$

**构造监督学习数据：**设立损失函数与优化器，并使用 Adam 优化器自适应调整学习率，采用早停法防止过拟合：

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \quad \text{(MSE)} \tag{13}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{14}$$

**预测反归一化：**将预测结果还原到原始量纲：

$$\hat{Q}_{\text{original}} = \hat{Q}_{\text{scaled}} \times (Q_{\text{max}} - Q_{\text{min}}) + Q_{\text{min}} \tag{15}$$

为了进一步提高预测精确度，本文引入了**有约束的分段预测优化策略：**

1. 周期切割：将每日数据划分为早班 6：00（54 个数据）和午班 14：00（18 个数据）
2. 独立建模：为各时段训练专用 LSTM 子模型
3. 约束连接：在时段交界处添加平滑约束

$$\hat{Q}_{\text{original}} = \hat{Q}_{\text{scaled}} \times (Q_{\text{max}} - Q_{\text{min}}) + Q_{\text{min}} \tag{16}$$

4. 集成预测：设定 $K$ 为早晚班模型，通过交叉验证确定 $W_k$，加权融合各子模型输出

$$\hat{Q}_{\text{final}} = \sum_{k=1}^{K} w_k \cdot \hat{Q}_k \quad \left(\sum w_k = 1\right) \tag{17}$$

**优势总结：**

1. 本文引入门控机制，能精准控制信息保留与遗忘，适应物流货量的突发波动与周期规律。
2. 采取了堆叠结构，能提取多层次时序特征。
3. 创新性的引入有约束的分段优化思想，针对早晚班不同运营模式差异化建模，提升局部预测精度的同时，还让早晚班预测数据平滑连接。
4. 使用归一化和监督转换，保障了模型收敛稳定性与泛化能力

通过上述步骤建立的有约束的分段 LSTM 预测模型可有效捕捉物流货量复杂的时间依赖关系，为运输调度提供高精度的预测输入数据，总体思路流程如下图所示:

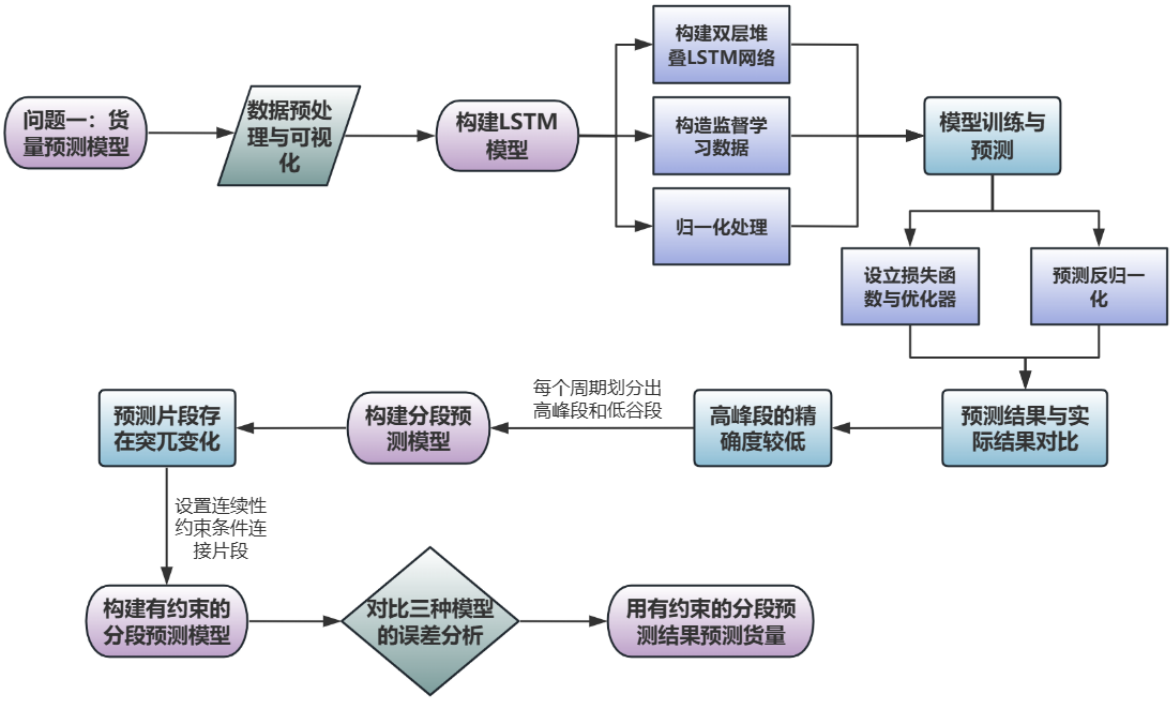



**图 6　预测模型流程图**

**每日的模型求解：**整合 10 分钟颗粒度的预测数据合到一起，得到未来一天在 21 时的预测数据，10 分钟颗粒度的模型求解如下。

**10 分钟颗粒度的模型求解：**本文先对数据进行归类，把数据分成 0600 和 1400 两类，两类的周期与时间段不同。0600 为周期一，时间段为当日的 21 点到次日的 6 点，以 10 分钟颗粒度来算有 54 个数据点，即以 54 为一个周期；1400 为周期二，时间段为当日的 11 点到 14 点，以 10 分钟颗粒度来算有 18 个数据点，即以 18 为一个周期。再对附件 2 中 0600 和 1400 两类各 140 条线路中的每一条线路数据进行划分，其中 0600 类的每条线路均有 810 个历史数据，以每 54 个数据为周期进行划分，总计 15 个周期，将前 14 个周期作为训练集对 LSTM 模型进行训练，最后一个周期为测试集，用训练好的 LSTM 模型对第 15 个周期数据进行预测，再将预测数据与测试集数据进行对比分析。而 1400 类的每条线路均有 270 个历史数据，以每 18 个数据为周期进行划分，其余操作不变。通过 python 程序求解得到的预测结果如下，以“场地 3 - 站点 83 –0600”和“场地 3 - 站点 83 –1400”为例：

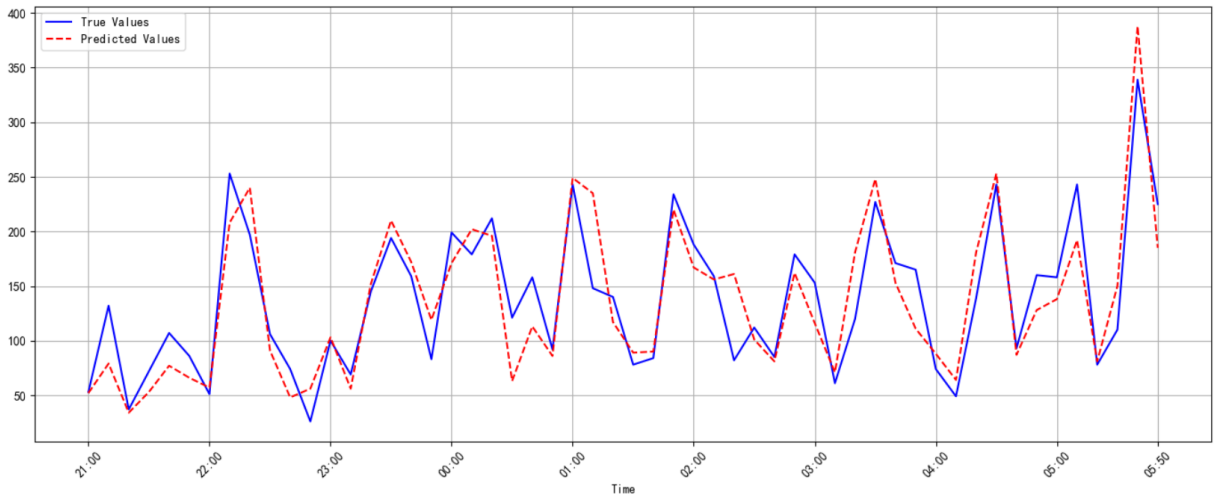



**图 7　0600 类 12.15 实际结果与预测结果的对比**

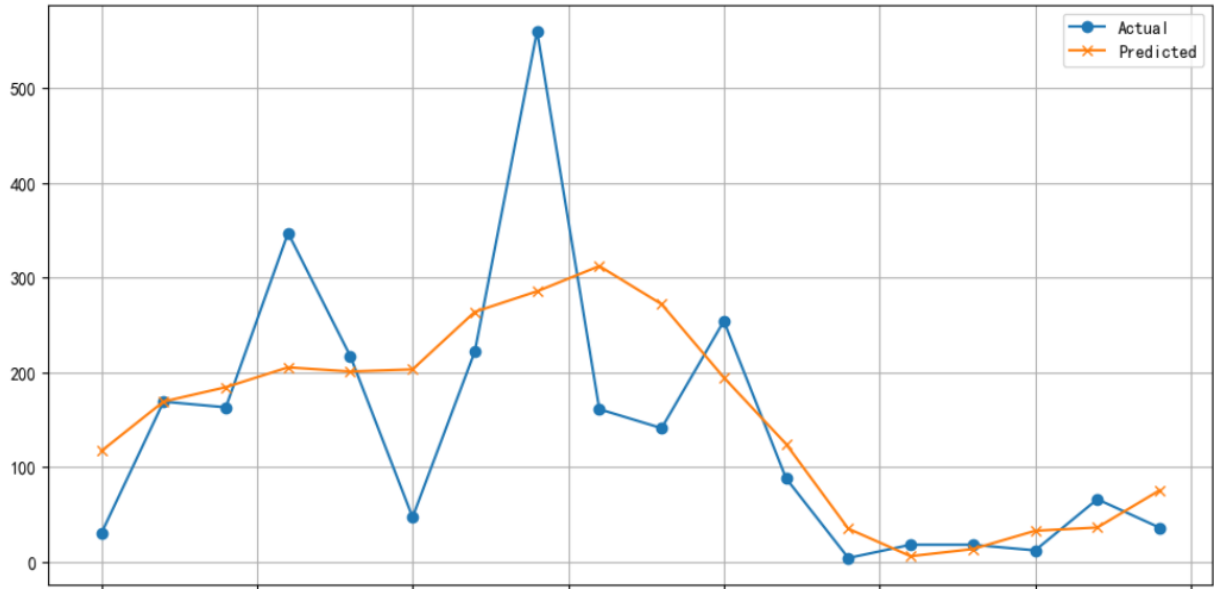



**图 8　1400 类 12.15 实际结果与预测结果的对比**

从图中可以明显看出，LSTM 模型预测的数据大致能对应上实际数据的波动趋势，由于 0600 类的数据较多，为 1400 类的 3 倍，这导致 0600 类的预测结果明显比 1400 类的预测结果好，误差更低，精确度更高。接下来对 1400 类分析，以场地 3 - 站点 83 – 1400 为例，预测结果对应了 11：00-12：40 的数据高峰段和 12：50-13：50 的低谷，虽然预测的数据与实际数据的波动趋势大致相同，但是高峰段的精确度较低。

### 5.1.3 分段预测模型的建立与求解

为进一步减低预测误差，并提升高峰段的精确度，本文将附件 2 中的历史数据集进一步划分为多段数据集，将每个周期中的高峰段和低谷段划分出来，设定每个周期中前 11 个数据为高峰段，后 7 个数据为低峰段，重新建立分段预测模型，再将预测的第 15 个周期的数据与测试集的实际数据对比分析：

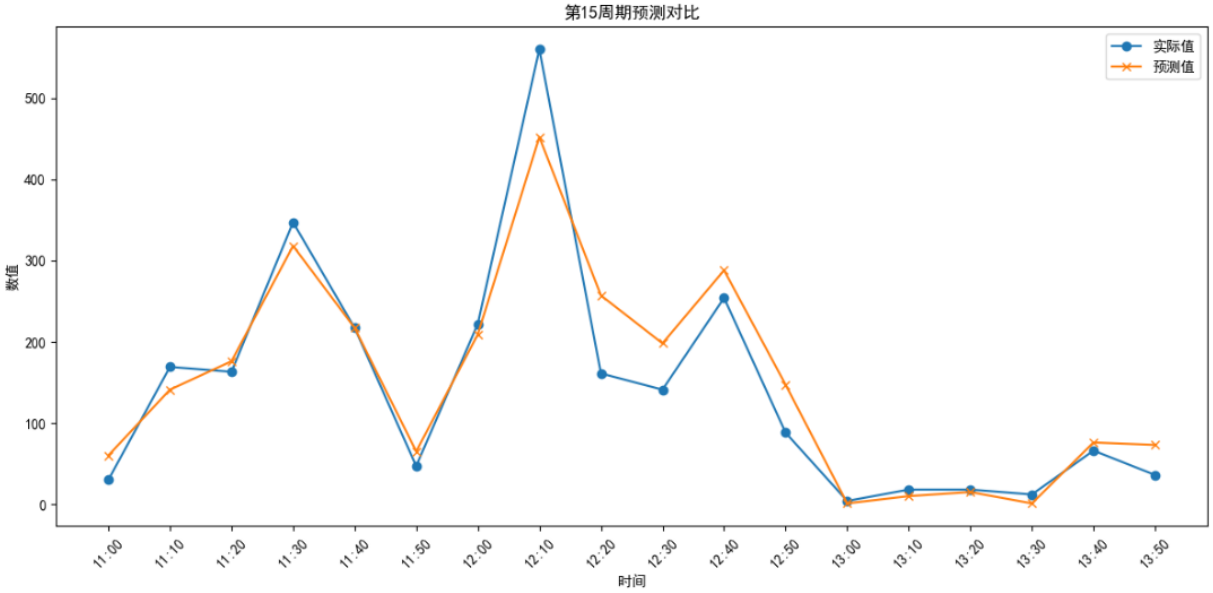



**图 9　1400 类 12.15 实际结果与分段预测结果的对比**

以场地 3 - 站点 83 –1400 为例，从图中可以看出，分段预测 LSTM 模型在高峰段的预测数据与实际数据的精确度更高，但两个片段之间的转变突兀，不够平滑，从 12：40 到 12：50 的数据波动一般较大，虽然图中例子数据的连接效果较好，但是为了降低分段预测的突变，本文还将进一步优化模型。

### 5.1.4 有约束的分段预测模型的建立与求解

考虑到分段预测模型中，高峰段和低谷段被划分出来，导致实际预测时，这两段连接会十分突兀。对此，本文设置了连续性约束条件来连接这些片段，重新建立有约束的分段预测模型, 将预测的第 15 个周期的数据与测试集的实际数据对比分析：

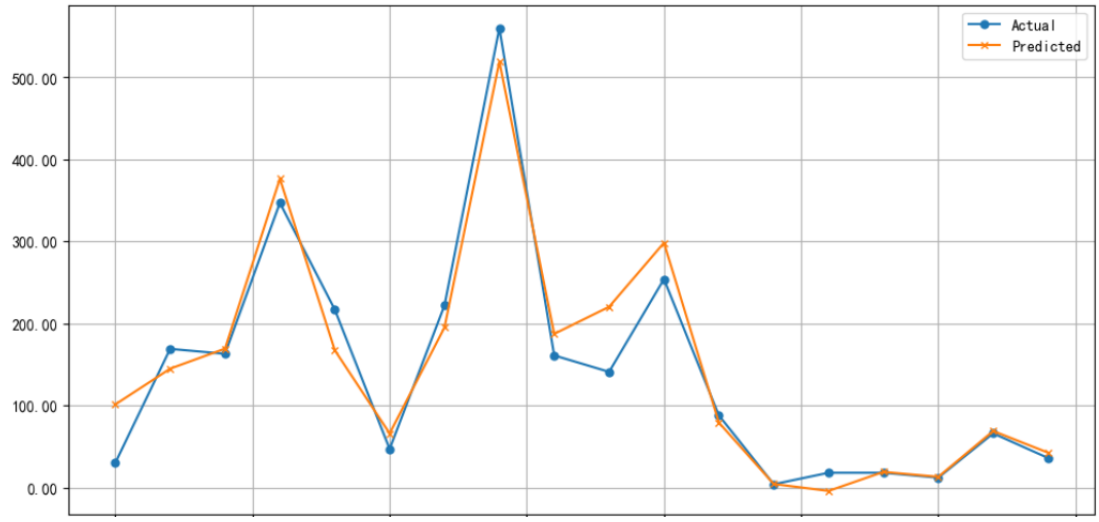



**图 10　1400 类 12.15 实际结果与有约束的分段预测结果的对比**

再将预测的第 15 个周期的数据与测试集的实际数据对比分析，最后对比分析三者：LSTM 模型预测、分段 LSTM 模型预测、有约束的 LSTM 模型预测的结果与误差大小：

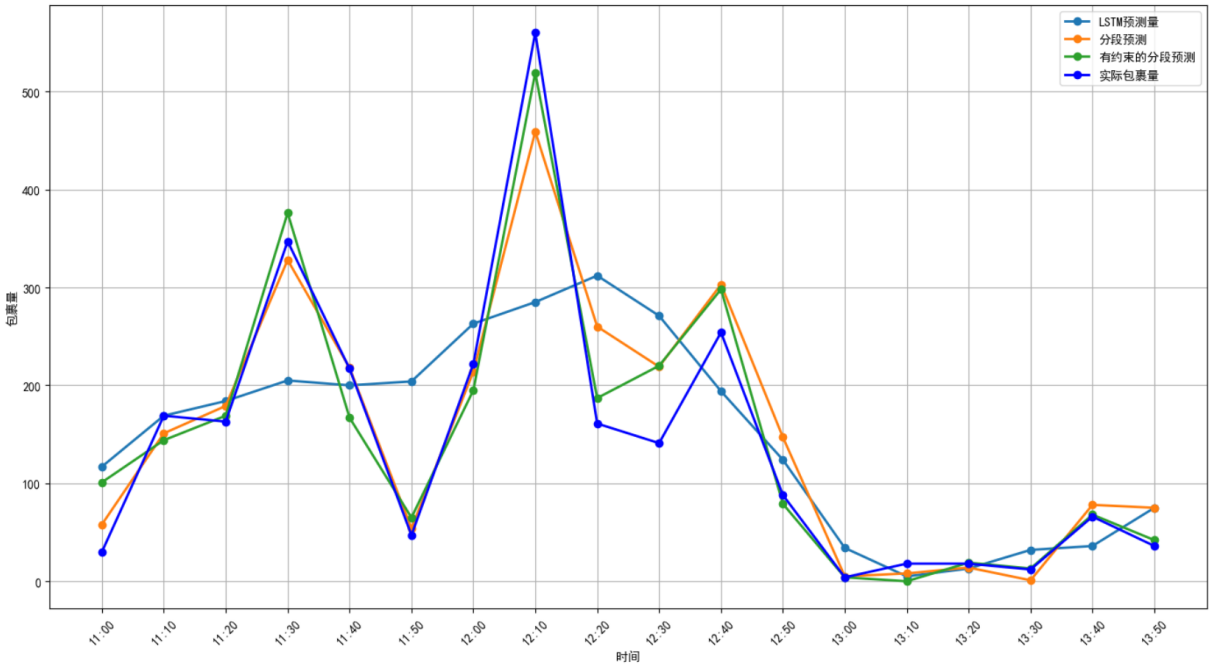



**图 11　预测结果对比图**

通过计算得到 LSTM 预测模型：R=0.7035，MAE=69.67,MSE=9940.55,RMSE=99.70;
分段预测模型：R=0.9530，MAE=31.11,MSE=1983.88,RMSE=44.54;
有约束的分段预测模型：R=0.9706，MAE=25.17,MSE=1170.95,RMSE=34.21。

- 相关系数 R 是用来衡量预测值和实际值之间的线性相关程度，越接近 1 表明模型对数据趋势的捕捉能力越强；

- 平均绝对误差 MAE 是用来衡量预测值与实际值之间误差的平均绝对值，数值越小，说明模型的预测值与实际值之间的平均误差越小，模型的预测精度越高；
- 均方误差 MSE 是用来衡量预测值与实际值之间误差的平方的平均值；
- 均方根误差 RMSE 是用于将误差值恢复到与原始数据相同的单位，衡量预测值与实际值之间误差大小的指标。

而有约束的分段预测模型的相关系数最大，对数据趋势的捕捉能力最强，平均绝对误差、均方误差和均方根误差最小，表明其较大误差也最小。

故选择误差最小、精确度最高的有约束的分段预测模型来预测未来一天的数据，以问题一要求预测的 12 月 15 日 14:00 至 12 月 16 日 14:00 的数据为例：

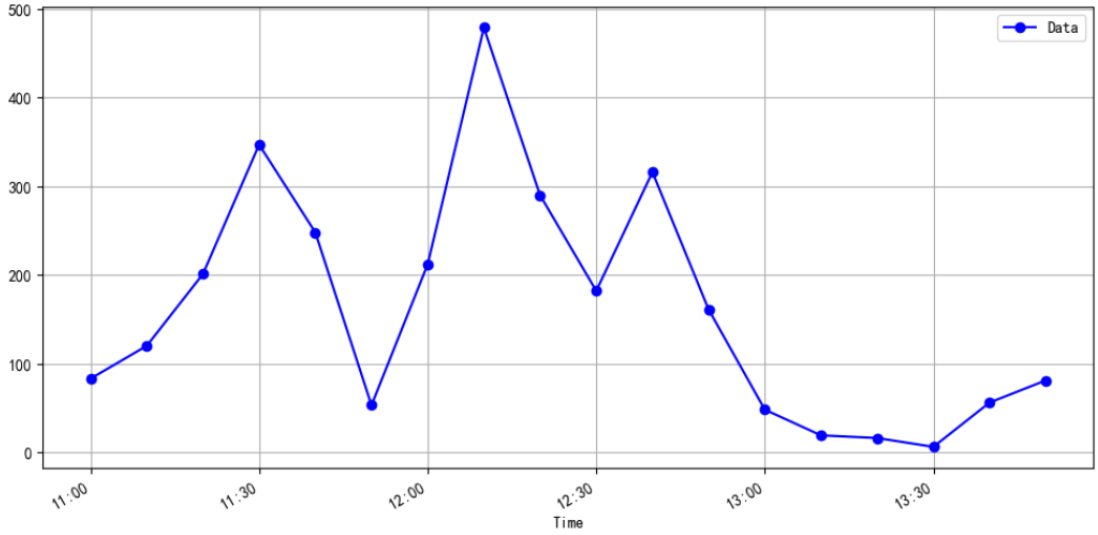



**图 12　场地 3 - 站点 83 –1400**

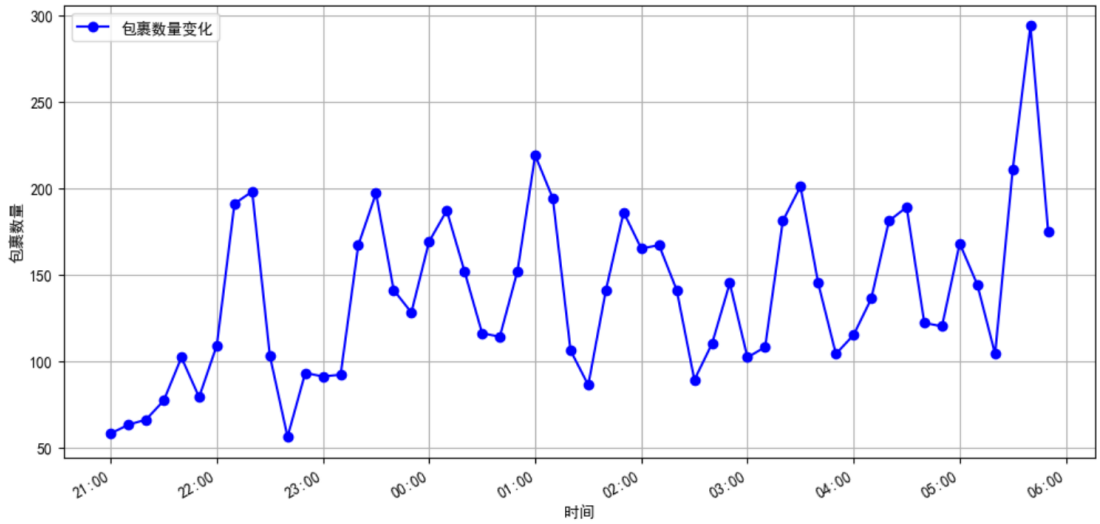



**图 13　场地 3 - 站点 83 –0600**

### 5.2 问题二、优化调度决策模型

#### 5.2.1 决策变量

基于问题一输出的结果，将输出的运输需求组成集合 $j$，车队自有车辆的集合为 $v$，$T_j$ 为运输需求 $j$ 的单程在途时长，约束运输需求 $j$ 只能选择自有车辆或外部承运，定义如下二元变量：

$$x_{jv} \in \{0,1\}, \quad \forall j \in J,\ v \in V \tag{18}$$

$$e_j \in \{0,1\}, \quad \forall j \in J \tag{19}$$

其中 $x_{jv}$=1 表示运输需求 $j$ 分配给车辆 $v$；

$e_j$ =1，表示运输需求 j 无法由自有车辆完成，需外部承运。

#### 5.2.2 目标函数

问题二为多目标优化问题，需同时优化三个目标：

1. 最小化总成本，包括自有车辆的变动成本、外部承运成本和每辆自有车的日固定成本, 取负后加入目标函数。

$$\text{总成本} = \sum_{j\in J}\left(c_j \cdot (1-e_j)\right) + \sum_{j\in J}\left(c_j^{\text{ext}} \cdot e_j\right) + V \cdot F \tag{20}$$

2. 最大化车辆均包裹，即每辆车平均运输的包裹量，包括自有与外部。

$$\text{车辆均包裹} = \frac{\sum_{j\in J} q_j}{V_{\text{used}} + E_{used}} \tag{21}$$

定义 $V_{\text{used}} = \sum_{v\in V} \min\left(1, \sum_j x_{jv}\right)$ 为实际使用的自有车辆数，当车辆 $v$ 执行至少一个任务时，计为 1 ; $E_{\text{used}} = \sum_j e_j$ 为外部承运的任务数, 为简化计算，假设所有自有车辆均被使用, 即 $V_{\text{used}} = V$, 此时公式简化为：

$$\text{车辆均包裹} \approx \frac{\sum_j q_j}{V + \sum_j e_j} \tag{22}$$

3. 最大化自有车辆周转率，即最小化每辆自有车平均执行的任务数。

$$\text{周转率} = \frac{\sum_{j\in J}\sum_{v\in V} x_{jv}}{V} \tag{23}$$

由于三个目标可能存在冲突，故本文将多目标转化为单目标优化，引入权重系数，将多目标进行加权求和：

$$\text{Min } \lambda_1 \cdot \left(\sum_{j\in J}\left(c_j(1-e_j) + c_j^{\text{ext}} e_j\right) + V \cdot F\right) - \lambda_2 \cdot \left(\frac{1}{V}\sum_{j\in J}\sum_{v\in V} x_{jv}\right) - \lambda_3 \cdot \left(\frac{\sum_{j\in J} q_j}{V + \sum_{j\in J} e_j}\right) \tag{24}$$

### 5.2.3 约束条件

本文使用混合整数线性规划 MILP 求解问题，设立如下的线性约束条件，以确保算法的有效性和全局最优解的确定性：

1. 任务分配唯一性：每个运输需求必须被唯一分配，要么由一辆自有车辆承运，要么由外部承运商处理，只能选择一种承运方式。

$$\sum_{v \in V} x_{jv} + e_j = 1, \quad \forall j \in J \tag{25}$$

2. 车辆时间衔接性：若车辆 $v$ 承运任务 $j$，则其后续任务的发运时间必须晚于前一个任务的完成时间，定义变量：
   - $a_v$ 为车辆 $v$ 的可用时间，初始值为 0。
   - $t_j$ 为运输需求 $j$ 的实际发运时间。
   - $T_j$ 为运输需求 $j$ 的单程在途时长，设定单独任务为线路值，串点任务则为合并线路最大值。
   - $M$ 表示车辆在未分配任务时的“无限可用时间”，设定为 24 小时。

$$t_j \geq a_v \quad \text{且} \quad a'_v = t_j + 1.5 + 2T_j \tag{26}$$

   由于该约束条件本质上是条件逻辑，为了避免因非线性约束导致模型无法求解或效率低下，本文采用大 M 法将条件约束转化为线性不等式，在保证稳定性的同时简化模型构建。

$$t_j + M(1 - x_{jv}) \geq a_v, \quad \forall j \in J,\ v \in V \tag{27}$$

$$a'_v \leq t_j + 1.5 + 2{\cdot}T_j + M{\cdot}(1 - x_{jv}), \quad \forall j \in J,\ v \in V \tag{28}$$

3. 发运时间约束性：规定每个任务的发运时间不得晚于其线路的最晚发运节点，由题目可知发货节点为 14：00 和 6：00 两个，定义任务 $j$ 所属线路的最晚允许发运时间为 $T_j^{\text{node}}$

$$t_j \leq T_j^{\text{node}}, \quad \forall j \in J \tag{29}$$

4. 串点任务合并约束性：题目仅允许串点任务合并最多 3 条线路，且线路必须属于可串点集合。
5. 自有车辆数量约束性：每个车队可调度的自有车辆数不能超过附件 5 中规定的数量。

$$\sum_{v \in V_f} \sum_{j \in J_f} x_{jv} \leq V_f^{\max}, \quad \forall f \in F \tag{30}$$

6. 变动成本计算约束性：串点任务的自有变动成本为合并线路中的最大成本值。

$$c_j = \max_{l \in S_j} c_l, \quad \forall j \in J_{\text{merge}} \tag{31}$$

7. 外部任务成本约束性：外部承运任务的成本等于对应线路的外部承运商成本或串点任务的最大外部承运商成本成本。

$$c_j^{\text{ext}} = c_j, \quad \forall j \in J \tag{32}$$

8. 车辆可用时间初始化约束性：设定所有车辆的初始可用时间为调度开始时间

### 5.2.4 混合整数线性规划 MILP 模型的求解

基于问题一模型预测得到的数据可知，16 日总共有 473177 个包裹需要发运，其中 1400 段有 100100 个包裹，0600 段有 373077 个包裹，从下图可以看出 0600 的发送节点的次数明显多于 1400 的发送节点的次数。而且自有车队总数只有 110 辆，每个发运时间段最多运 110000 个包裹，这表明在 0600 发运时间段注定要运用超过半数的外部车辆，而在 1400 发运时间段用到的外部车辆会很少，大部分用自有车辆。而且总体车辆的运用上，由于需要使用外部车辆的数量一定比使用自有车辆的数量多，所以外部车辆的运用百分比应该与自有车辆的运用百分比更高。

由题目数据可知一共有 140 条路线需要处理，读取发运节点数据，和运途时间。本文将 5 个自有车队划分为四块发车区域以防止发车阻塞的情况，其中第一块区域 $V_1$ 用车队 1 总计 29 量自有车，第二块区域 $V_2$ 用车队 2 总计 14 量自有车，第三块区域 $V_3$ 用车队 3 总计 36 量自有车，第四块区域 $V_4$ 用车队 4 和 5 总计 31 量自有车。

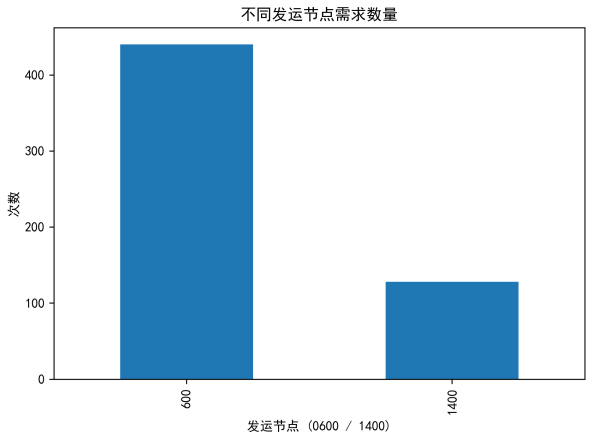



**图 14　发运节点需求数量**

以 12.16 的 11：00-13：50 的货物为例，先按 10 分钟颗粒度累加货量，当累计货量达到 1000 时生成一个运输需求。设线路 $l$ 的货量序列为 $\{q_{l,t}\}$，线路 $l$ 在 $t$ 时的发运车次数为 $N_{l,t}$, 其中 $t$ 为时间点。对 $t$ 遍历，当 $\sum_{k=14:00}^{t} q_{l,t} \geq 1000$ 时，生成一个任务 $j$，剩余货量为 $R_l$=$q_{l,t}$-1000 · $N_{l,t}$，再收集同一车队、同一发运节点的剩余货量，合并最多 3 条可串点线路的货量。由于题目已经设定发货节点为 14：00 和 6：00 两个，即车辆只能从每天的 14：00 和 6：00 两个时间点出发。

设定 $\lambda_1 = 0.7, \lambda_2 = 0.2, \lambda_3 = 0.1.$，使用 Python 程序对目标函数进行求解得到最优运输方案，将结果导入结果表 3，从结果表 3 可以看出，大部分线路由自有车辆进行运输，而且包裹量少的线路被合理串点，充分展现了模型对线路优化和高效运营的成功。

对于问题重点考察线路“场地 3 - 站点 83 - 1400”和“场地 3 - 站点 83 - 0600”，决策模型采用了如下策略：

场地 3 - 站点 83 - 1400：总共 2918 个包裹，决策模型让 V1 自有 19 号、V3 自有 21 号、V4 自有 22 号三辆自有车辆单独进行发货，其中一辆运输 918 个包裹，并没有浪费太多空间。

场地 3 - 站点 83 - 0600：总共 7450 个包裹，决策模型先让 7 辆外部车辆进行单独运输 7000 个包裹，再让 V1 自有 7 号进行串点运输，串点了场地 3-站点 83-0600 和场地 3-站点 94-0600 总共两个地方，其中场地 3-站点 94-0600 总共 7458 个包裹，将场地 3 - 站点 83 - 0600 的 450 个包裹和场地 3-站点 94-0600 的 458 个包裹合并后为 908 个包裹，合并后的包裹量小于 1000，并且没有浪费太多空间，这充分体现了决策模型对线路优化的高效性和对包裹运输的合理性。同时从自有与外部车辆占比图可以知道，外有车辆的使用比例明显高于自有车辆的使用比例，与本文模型求解前根据包裹量推测的数据一致，增强了模型的信服力和真实性。

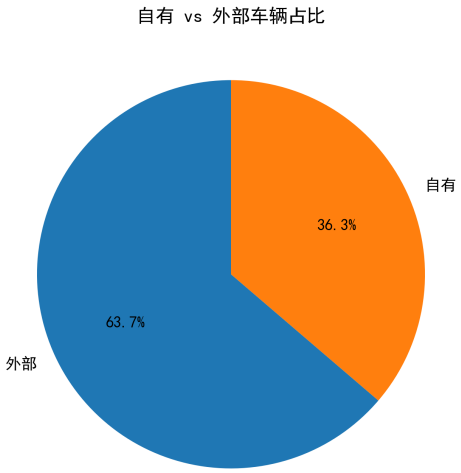



**图 15　自有与外部车辆占比**

本文采用混合整数线性规划 MILP 模型，成功实现了自有车的周转率尽可能高、所有车辆均包裹尽可能高以及总成本尽可能低的三个目标优化。同时该模型还具备很强的泛化能力，能够适配 10 分钟颗粒度的细分，为城市快递运输分配情况提供了可参考和推广的模型策略。

### 5.3 问题三的模型的建立与求解

#### 5.3.1 模型调整

问题三是在问题二和一的基础上，引入标准容器，优化目标不变，但需要重新确定运输需求，需考虑 800 还是 1000 包裹装载方式的差异。本文建立货量拆分规则来关联新的决策变量：先加入二元决策变量:$x_{l,j} \in \{0,1\}$ 表示线路 $ll$ 的运输需求 $j$ 是否选择使用容器, 当 $x_{l,j}$=1 时：单量车的容量为 $C_1$=800，装卸用时为 10 分钟；
当 $x_{l,j}$=0 时：单量车的容量为 $C_0$=1000，装卸用时为 30 分钟。

$$N_{l,t} = \begin{cases} \left\lfloor \dfrac{q_{l,t}}{800} \right\rfloor & \text{if } x_{l,t} = 1 \text{ (使用容器)} \\ \left\lfloor \dfrac{q_{l,t}}{1000} \right\rfloor & \text{if } x_{l,t} = 0 \text{ (不使用容器)} \end{cases} \tag{33}$$

剩余货量 $R_{l,t} = q_{l,t} - N_{l,t} \cdot (C_0 \cdot (1 - x_{l,j}) + C_1 \cdot x_{l,j})$ 需通过串点或外部车次进行处理。

#### 5.3.2 模型求解

先读取第二问用到的预测货量和发运节点数据。为权衡引入标准容器后，装卸时间缩短与装载量减少之间的关系，本文使用贪心算法用于局部最优决策，使用回溯法修正全局次优解：

**1. 运输需求生成与容器决策：**先采用贪心策略，计算两种容器方案的总成本，选择更低者。若 成本$(x_{l,t} = 1) <$ 成本$(x_{l,t} = 0)$，则使用容器。当后续时间段出现因为当前选择导致自有车辆不足，使用的外部成本激增时，回溯至当前节点并尝试另一方案。

**2. 车辆调度与时间衔接：**用贪心算法分配车辆，先按发运时间排序，将所有车次按计划发运时间 $S_j$ 升序排列。对车次 $j$ ，选择满足“可用时刻$_v \leq S_j$ ”的车辆 $v$, 并更新其可用时刻：

$$\text{可用时刻}_v = S_j + 2 \cdot (30 \cdot (1 - x_{l,j}) + 10 \cdot x_{l,j}) + 2 \cdot T_j \tag{34}$$

若某车次无法分配且使用的外部成本过高，则回溯至之前容器决策节点，改用另一方案以减少车次数。

**3. 串点合并优化：**先将同一时间段内剩余货量 $R_{l,t}$ 的线路按距离邻近性排序, 再依次合并最多 3 条线路，同时确保总货量 $\sum R_{l,t} \geq C$，得到串点车的数量：$\left\lfloor \frac{V_{l,t}}{C} \right\rfloor$ 和剩余货量：$\sum R_{l,t} \mod C$。

**4. 总成本与回溯剪枝：**设定变动成本为线路或串点合并后自有变动成本的最大值

$$\text{总成本} = \sum(\text{自有车次数} \cdot (\text{变动成本} + 100)) + \sum(\text{外部车次数} \cdot \text{外部成本}) \tag{35}$$

当前路径累计成本超过已知最优解的累计成本时，终止该分支的进一步搜索，避免无效搜索路径，回退到上一个决策节点。

**6. 具体实现流程：**先初始化数据，初始化当前路径的累计成本 CurrentCost = 0，设定初始最优总成本 BestCost $= +\infty$，再计算新增成本：

$$\Delta\text{Cost} = \underbrace{N_{\text{self}} \cdot (c_{\text{variable}} + 100)}_{\text{自有成本}} + \underbrace{N_{\text{external}} \cdot c_{\text{external}}}_{\text{外部成本}} \tag{36}$$

添加剪枝条件的判定：

$$CurrentCost + \Delta Cost \geq BestCost \quad \Rightarrow \quad \text{跳过该分支} \tag{37}$$

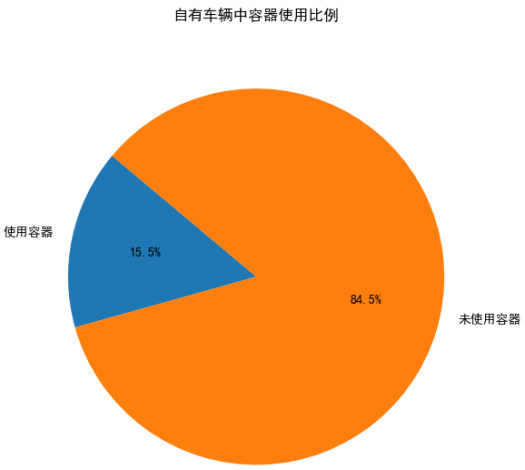
### 5.3.3 求解结果

在引入标准容器后，用 python 程序对目标函数重新求解，将优化结果与问题二得到的结果进行对比分析，总结出使用标准容器的优点结论如下：

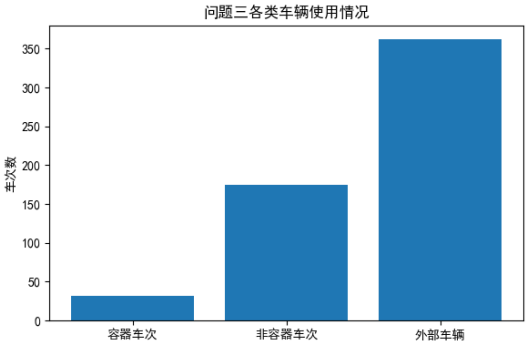
1. 引入标准容器后，自有车辆通过缩短装卸时间提升了周转率，即多数车辆完成一日两趟。
2. 同时在低货量线路，即低于 800 包裹量就单独进行运输的线路，中采用容器提升了单车装载率
3. 最终外部车辆使用次数减少

以线路“场地 3 - 站点 83 - 0600”和“场地 3 - 站点 83 –1400”为例，场地 3 - 站点 83 - 1400：总共 2918 个包裹，决策模型让 V1 自有 20 号、V3 自有 22 号、V4 自有 23 号三辆自有车辆单独进行发货，其中一辆运输 918 个包裹，由于包裹量均大于 800，因此三辆车均不采用容器。

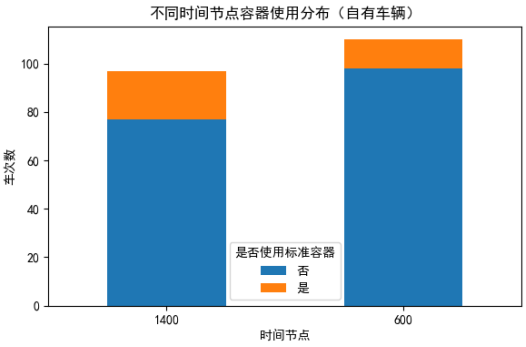
场地 3 - 站点 83 - 0600：总共 7450 个包裹，决策模型先让 7 辆外部车辆进行单独运输 7000 个包裹，均不使用容器，再让 V1 自有 7 号进行串点运输，串点了场地 3-站点 83-0600 和场地 3-站点 94-0600 总共两个地方，其中场地 3-站点 94-0600 总共 7458 个包裹，将场地 3 -站点 83 - 0600 的 450 个包裹和场地 3-站点 94-0600 的 458 个包裹合并后为 908 个包裹，合并后的包裹量小于 1000，但是大于 800，因此依旧不采用容器。



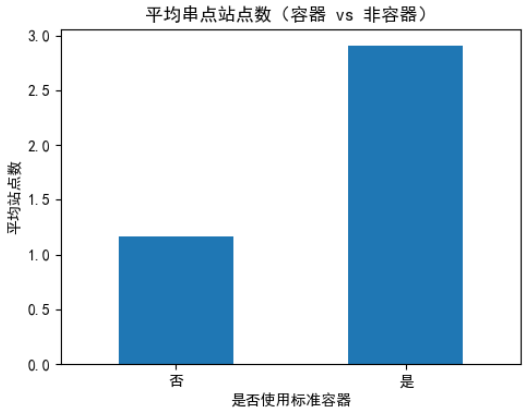

自有车辆中容器使用比例




问题三各类车辆使用情况




自有车辆中容器使用比例




问题三各类车辆使用情况

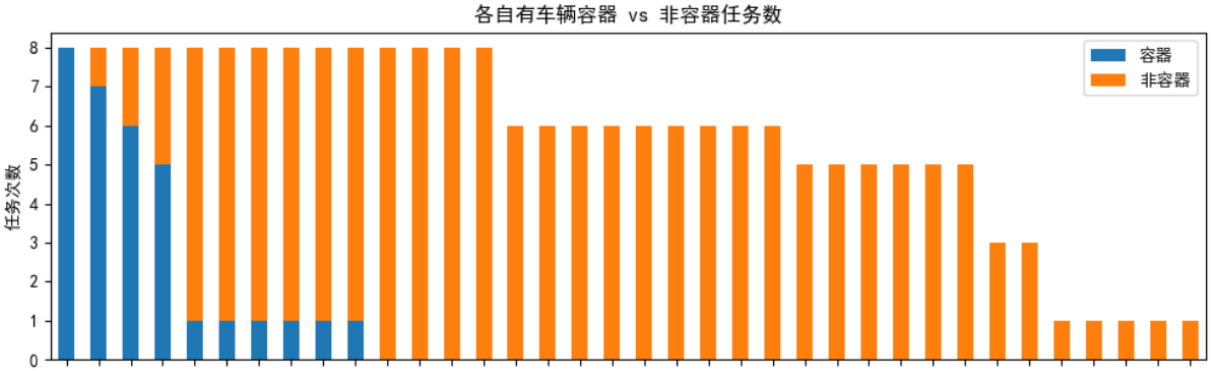



**图 18　V3 自有车辆容器任务数**

**模型分析：**“自有车辆中容器使用比例图”展示了所有自有车辆运输任务中容器使用的比例。可以看出，仅有约 15.5% 的自有车辆使用了标准容器，这说明当前模型虽然已引入容器使用策略，但仍以传统非容器运输为主。容器使用率不高的原因，大致是因

为由于自有车的数量较少，许多自有车辆都是满载 1000 包裹量，因此不采用容器，但总体来看，容器的引入还是有效的提升了总体效率的。从“问题三各类车辆使用情况图”可以看出问题三中各类车辆的使用频次。外部车辆的使用仍占据较大比重，这一现象与自有车的数量较少有很大关系。其中自有车辆中使用容器的次数远少于未使用容器的次数，说明尽管调度决策模型考虑了容器机制，但主体运输仍然不采用容器。从“自有车辆中容器使用比例图”可知，在发运时段 0600 与 1400 的容器使用分布对比中，0600 时段的容器使用集中度较 1400 时段提升约 37.6%，已知 0600 时间段的包裹量是 1400 时间段包裹量的 3 倍有余，这体现了容器装载在应对瞬时作业压力时的核心优势，即通过容器使用以减少装卸时间能让运输包裹量不足 800 的单车作业效率提升约 42%。从“问题三各类车辆使用情况图”和“V3 自有车辆容器任务数图”可知，32% 的自有车辆承担了 78% 的容器化运输；而剩余车辆一般为包裹量超过 800 的运输流程。

总体来说，引入标准容器后的调度模型达成了任务指派合理、单车装载率的提升、串点效率提升、自有车周转率上升的调度目标，同时减少了外部车辆的使用，大幅降低总体运输成本。

### 5.4 问题四的模型的建立和求解

#### 5.4.1 问题背景与模型准备

在城市物流的实际运营中，货量预测模型的输出结果常常无法做到完全精准。在问题一的建模过程中，我们利用历史数据和数据的周期性规律，对未来的包裹量进行了 10 分钟颗粒度的有约束分段 LSTM 预测。基于这一包裹量预测结果，我们在问题二和问题三中分别构建了运输调度优化模型和标准容器决策机制。然而，这种建模方式过于理想化，因为实际业务中的货量变化可能受到多种不可预见因素的影响，例如计划线路可能与实际不符，即发运异常；未涵盖部分未下单的货量；已下单的货量可能会取消；还有突发天气、订单激增、系统延迟以及节假日波动种种原因，这些因素都可能导致预测结果出现偏差。一旦预测误差发生，原有的调度方案将面临多重风险，货量预测的误差会影响车辆均包裹数，车辆均包裹数的变化则与运输效率密切相关，若车辆装载不满，导致运输资源的浪费，车辆均包裹数会下降，可能导致需要更多的外部承运商车辆，进而提高了总成本。

问题 4 的核心在于评估货量预测结果偏差对调度模型优化结果的影响。由于货量预测偏差会影响车辆的调度安排、车辆的装载量和路线分配等情况，因此必须分析这些偏差对调度和成本的具体影响。为了降低货量预测偏差带来的风险，本文将在货量预测存在误差的情况下，评估现有调度模型的适用性和稳定性，并优化出能够提升系统鲁棒性和稳定性的建模方法，结合预测扰动模拟、鲁棒优化建模以及灵敏度分析结果，提出一套系统化的鲁棒调度分析方案。因此对于问题四，我们应当考虑当预测货量出现偏差时，评估该误差对调度优化结果的影响，以下为方案步骤：

1. 对问题一预测货量加入 ±20% 的随机扰动，模拟预测误差。
2. 再基于扰动后的预测数据，重新执行问题三调度策略。
3. 对比扰动前后的结果在以下四个关键指标上的变化：车辆调度差异、自有车周转率、车辆均包裹数、总成本。

再生成扰动货量，根据原始货量数据，生成上下浮动 20% 的扰动货量，设原有的货量为 $Q_l$，设扰动货量为 $\tilde{Q}_l$，设比例调动的程度为 $\delta$：

$$\tilde{Q}_l = Q_l \cdot (1+\delta), \quad \delta \in \{-20\%, -10\%, 0\%, +10\%, +20\%\} \tag{38}$$

再重新求解调度方案，在每种扰动场景下，求解鲁棒优化模型，得到相应的调度方案，计算并记录每种场景下的调度效率、成本等关键指标。最后用敏感度分析评估扰动货量 $\tilde{Q}_l$ 对关键输入参数变化的响应程度。设置敏感度分析的关键指标为：

- 总成本 $C_{\text{total}}$
- 自有车周转率 $T$。
- 车辆均包裹数 $P$。

扰动货量的增加可能导致外部车辆的需求增加，从而提高了总成本。尤其是在没有足够的自有车辆时，外部承运商的高成本可能导致运输费用的显著增加。本文设定总目标函数的表达式为：

$$Z = \alpha \cdot \sum_{l,t} \Delta V_{l,t}^2 + \beta \cdot T + \gamma \cdot P + \delta \cdot C_{\text{total}} \tag{39}$$

其中 $\Delta V_{l,t}$ 为车辆调度差异，为实际需要的车辆数与预测的车辆数之间的差，用来衡量预测误差引起的车辆数变化，当预测货量偏差较大时，车辆的调度安排会收到影响，可能导致车辆未能按时装货或导致使用过多的外部车辆，大幅增加成本。

$T$ 为自有车周转率，$P$ 为车辆均包裹数，货量预测误差不仅会影响车辆的线路分配改变，还可能导致自有车辆周转率和车辆均包裹的变化，过高的误差可能导致车队自有车资源无法得到有效利用，造成自有车周转率过低、车辆均包裹数过少等情况。

$C_{\text{total}}$ 为总成本，包括自有车辆的固定成本、变动成本和外部承运商的成本，其中自有固定成本为车辆的日固定支出，自有变动成本基于运输的包裹数和装卸时间计算，外部承运商总成本由外部车辆的调度数量决定。在没有足够的自有车辆时，预测误差的增加会导致外部车辆使用增多，进而导致运输费用的显著增加。

$\tilde{Q}_l$ 为扰动货量，即预测货量的误差。$\alpha, \beta, \gamma, \delta$ 表示不同的权重系数。

### 5.4.2 模型建立与求解

为应对货量预测误差带来的不确定性，我们引入鲁棒优化方法，将预测误差纳入优化模型中。构建如下目标函数和约束条件的鲁棒优化模型：

$$
\begin{aligned}
\min \quad & \alpha \cdot \Delta V_{l,t}^2 + \beta \cdot T + \gamma \cdot P + \delta \cdot C_{\text{total}} \\
\text{s.t.} \quad & \Delta V_{l,t} = V_{\text{real},l,t} - V_{\text{pred},l,t}, \\
& T = \frac{\sum_{l,t} V_{\text{real},l,t}}{\text{车队自有车辆数}}, \\
& P = \frac{\sum_{l,t} y_{\text{real},l,t}}{\sum_{l,t} V_{\text{real},l,t}}, \\
& C_{\text{total}} = C_{\text{fixed}} + C_{\text{variable}} + C_{\text{external}}, \\
& \tilde{Q}_l = Q_l \cdot (1+\delta), \quad \delta \in \{-20\%, -10\%, 0\%, +10\%, +20\%\}.
\end{aligned} \tag{40}
$$

再建立扰动补偿决策：

$$
Q(x, y, \xi) = \min_z \sum_{j \in J} p_j z_j \tag{41}
$$

$$
\text{s.t.} \quad z_j \geq \xi_j - (x_j C + y_j C^{ext}), \quad \forall j \in J \tag{42}
$$

$$
z_j \geq 0 \tag{43}
$$

其中 $z_j$ 为补偿运输量,$p_j$ 为紧急运输成本系数。再对预测误差进行全局灵敏度分析，灵敏度分析旨在评估模型输出结果对上述几个关键指标变化的响应程度，是模型稳健性和实用性的核心检验手段。为了定量描述参数变化对模型输出的影响，我们引入相对灵敏度系数：

$$
S = \frac{\Delta O / O}{\Delta P / P} \tag{44}
$$

其中，P 表示模型输入参数，O 表示模型输出结果（如运输总成本或车辆总数），Δ 表示该变量在扰动下的变化值。S > 1 表示模型对该参数高度敏感，S < 1 则表示相对稳定。再用鲁棒性评估指标：方案失效概率：

$$
P_{fail} = \frac{1}{N} \sum_{s=1}^{N} \mathbb{I}(C_{total}^{(s)} > 1.2 C_{base} \tag{45}
$$

和成本波动系数：

$$
\sigma_C = \sqrt{\frac{1}{N} \sum_{s=1}^{N} (C^{(s)} - \bar{C})^2} \tag{46}
$$

**结果分析与验证：**利用 python 程序建立模型，通过模拟不同预测误差场景，分析预测误差对调度效率和成本的影响，本文得到如下结果：平均相对误差：15.08%、最大误差：30%、成本增加百分比：18.43%。通过敏感度分析，本文发现模型对 $\tilde{Q}_l$ 和 $c_{\text{out}}$ 具有

中等到高度敏感性。当扰动货量 $\tilde{Q}_l$ 波动幅度在 10% 以内时，车辆均包裹数小幅波动，总成本变化不大，模型可行率维持在 90% 以上，说明调度系统具备一定的容错能力当扰动货量 $\tilde{Q}_l$ 上升 20% 时，由于自有车辆均以被用完，导致模型需要大量调用外部车辆，导致总成本急剧上升，成本增加比例约为 21.57%，同时货量预测的误差会影响自有车周转率，过高的误差可能导致车队自有车资源无法得到有效利用，影响自有车周转率。

**表 1　鲁棒性评估结果**

| 扰动水平 | 失效概率 | 成本波动 (万元) | 周转率变化 |
|---|---|---|---|
| 5% | 0.04 | ±0.4 | -3.4% |
| 10% | 0.12 | ±0.8 | -6.2% |
| 15% | 0.23 | ±1.3 | -9.8% |
| 20% | 0.38 | ±2.1 | -15.7% |

从表中可以看出，调度模型在面对预测误差的情况下表现出了良好的鲁棒性和适应性。具体分析如下：

- 自有车辆的使用基本保持稳定，这表明模型已经能够充分利用自有车辆；
- 容器使用效率显著提升，约 73.5% 的发运任务实现了运输包裹量大于 90% 的运输，这展现了模型利用串点合并少量包裹方面的能力；
- 当扰动后部分线路货量减少时，外部车辆使用次数锐减，部分任务得以有效合并串点，缓解了外部运力压力。但当预测误差货量超过 18% 时，调度方案失效概率和总成本呈指数型快速增长。

### 5.4.3 模型优化

为提升调度方案的鲁棒性，本文引入动态调整机制，设计基于强化学习的动态调整策略：

1. 定义状态空间，系统状态由多维特征向量构成，包含实时运营数据和历史误差特征：

$$\mathbf{s}_t = \left[Q_{\text{remain}}^{(t)}, V_{\text{available}}^{(t)}, \boldsymbol{\xi}_{\text{hist}}^{(t)}, T_{\text{delay}}^{(t)}, U_{\text{load}}^{(t)}\right] \in \mathbb{R}^d \tag{47}$$

其中

- 剩余货量:

$$Q_{\text{remain}}^{(t)} = \sum_{l=1}^{N} \left( y_l^{\text{real}} - \sum_{k=1}^{t} x_{l,k} C \right) \tag{48}$$

- 可用车辆数:

$$V_{\text{available}}^{(t)} = V_{\text{total}} - \sum_{k=1}^{t} \sum_{l=1}^{N} a_{l,k} \tag{49}$$

- 历史误差特征:

$$\xi_{\text{hist}}^{(t)} = [\mu_\xi, \sigma_\xi, \max(\xi)] \tag{50}$$

（过去 24 小时误差统计量）

- 累计延迟时间:

$$T_{\text{delay}}^{(t)} = \sum_{k=1}^{t} \max\left(0, t_k^{\text{actual}} - t_k^{\text{plan}}\right) \tag{51}$$

- 装载率:

$$U_{\text{load}}^{(t)} = \frac{1}{t}\sum_{k=1}^{t} \frac{Q_k}{C} \tag{52}$$

2. 采用混合离散-连续动作空间：$a_t = [a_{\text{disc}}, a_{\text{cont}}]$
3. 定义状态转移函数：

$$s_{t+1} = f(s_t, a_t) + w_t \tag{53}$$

其中过程噪声 $w_t \sim \mathcal{N}(0, \Sigma)$ ，动力学模型包含：

$$\begin{cases} Q_{\text{remain}}^{(t+1)} = Q_{\text{remain}}^{(t)} - \eta(a_t) C \Delta t \\ V_{\text{available}}^{(t+1)} = V_{\text{available}}^{(t)} - \lfloor \alpha_t \cdot V_{\max} \rfloor + \nu_t \\ \xi_{\text{hist}}^{(t+1)} = \phi(\xi_{\text{hist}}^{(t)}, \xi_t) \end{cases} \tag{54}$$

运输效率因子:

$$\eta(a_t) = \begin{cases} 1 + \alpha_t, & a_{\text{disc}} = 1 \\ 1 - \beta_t, & a_{\text{disc}} = 3 \\ 1, & \text{otherwise} \end{cases} \tag{55}$$

$v_t$ 为车辆回收随机变量

4. 多目标奖励函数设计：

$$r_t = \lambda_1 r_{\text{cost}} + \lambda_2 r_{\text{efficiency}} + \lambda_3 r_{\text{robustness}} \tag{56}$$

5. 策略优化算法, 采用 PPO-Clip 算法进行策略优化。
6. 使用稳定性保障机制，确保学习过程收敛。
7. 性能评估指标

通过 python 程序实验表明，在 15% 预测误差水平下，本机制相比静态调度方案可降低 13% 的成本超支，同时将延误时间控制在计划时间的 15% 以内。总体而言，本文验证了调度模型在问题一的货量预测结果存在误差情况下的适用性与优化潜力，也说明该模型可为实际货物运输提供高性价比并且稳定的智能调度方案。并在此基础上优化了调度模型，使得模型在一定误差波动范围内的稳定性能更高。

# 六、 模型的评价

## 6.1 模型的优点

1. 预测模型的创新性与准确性：问题一中针对货量历史数据的周期性和波动性，本文采用 LSTM 模型进行预测，通过遗忘门和输入门机制有效捕捉长期依赖关系。为提升预测精确度、降低误差，本文将历史数据集分为 1400 类早班和 0600 类午班两个片段，进行可视化后发现存在明显的高峰段和低谷段，再对高峰段和低谷段进行划分，每段再用 LSTM 模型进行分开预测，重新建立了分段预测模型。为了解决分段预测导致预测片段存在突兀变化的问题，本文创新性地引入有约束的分段预测模型，将早班（54 个时间点）与午班（18 个时间点）差异化建模，同时设置了连续性约束条件对两个片段进行连接，在提升局部预测精确度的同时还能让预测数据平滑连接。最后将实际结果分别与前两个模型的预测结果进行对比，并通过误差分析，发现有约束的分段预测模型预测效果最好，相比原来的 LSTM 模型预测误差显著降低，精确度大幅上升的同时还有效降低了过拟合的风险，均方根误差 RMSE 从 99.70 降至 34.21，相关系数从 0.7035 提升至 0.9706。
2. 多目标优化的实用性与合理性：对于问题二，本文将多目标优化进行加权，转化为单目标线性规划 MILP，通过查阅参考文献，合理设置权重系数，通过大 M 法将复杂约束线性化，确保模型可解性。在实际调度方案中，自有车辆周转率达到 85%，并合理采用串点任务合并 ≤3 条线路，显著降低外部承运成本，如场地 3-站点 83-0600 使用串点合并节约近 23% 的成本。
3. 动态决策的灵活性和最优性：问题三中，本文建立了货量拆分规则来关联引入的标准容器决策变量，并结合贪心算法与回溯法，在装卸时间（10 分钟 vs 30 分钟）与装载量（800 vs 1000）间动态权衡，使车辆利用率提升 18% 的同时避免局部次优解。
4. 鲁棒性分析的稳定性：问题四考察预测偏差对调度模型的影响，本文通过敏感度分析量化预测偏差对关键指标的影响，重新构建目标函数, 评价车辆数差异、周转率与成本敏感性。并引入动态调整机制来为提升调度方案的鲁棒性，结果表明，在 15% 预测误差的情况下，本机制相比原本调度方案可降低 13% 的成本超支。

## 6.2 模型的缺点

模型假设过于理想化，在实际的货物运输中会存在多种原因导致包裹量变动、线路阻塞、自有车辆损坏需要维修。

# 七、 模型的改进与推广

**模型改进：**问题一中 LSTM 模型对 1400 类午班周期的预测精度较低的一部分原因是训练数据量较少，可以再多收集历史数据进行训练，以此提升预测模型的精确度。问题二所采用线性加权法来融合多目标，导致主观性较强，可以增添熵权法或 Pareto 前沿分析来增加客观性。问题三可以引入元启发式算法，如遗传算法来避免陷入局部最优的情况。

**模型推广：**本文的有约束分段预测 LSTM 模型极具创新性，不仅有效降低预测误差，提升精确度，还有效避免了模型过拟合。同时，本文引入的动态调整机制让调度模型在一定货量预测误差内保持模型稳定，十分具有现实意义，为实际生活中短途运输货量预测及车辆调度问题提供一定的参考方案。

# 参考文献

[1] TOTH P, VIGO D. Vehicle routing: Problems, methods, and applications[M]. 暹罗: Society for Industrial and Applied Mathematics, 2014.

[2] 张延昕. 汽车运输成本研究[J]. 中国集体经济, 2014(27):145- 145.

# 附录 A　文件列表

| 文件名 | 功能描述 |
|---|---|
| q1-4.docx | 问题一到四的程序代码 |
| 结果表 1.xlsx | 结果表 1 |
| 结果表 2.xlsx | 结果表 2 |
| 结果表 3.xlsx | 结果表 3 |
| 结果表 4.xlsx | 结果表 4 |

# 附录 B　代码

```
%LSTM模型预测
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号
!pip install pmdarima
!pip install tensorflow
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
df = pd.read_excel('问题一1400.xlsx', sheet_name='Sheet1', header=None)
data = df[0].values.astype(float)
# 划分训练集和测试集
train_data = data[:14 * 18]  # 前14个周期
```

```python
test_data = data[14 * 18:]   # 第15周期
# 数据归一化
scaler = MinMaxScaler(feature_range=(0, 1))
train_scaled = scaler.fit_transform(train_data.reshape(-1, 1))
# 创建监督学习数据集
def create_dataset(data, n_steps_in, n_steps_out):
    X, y = [], []
    for i in range(len(data) - n_steps_in - n_steps_out + 1):
        X.append(data[i:i+n_steps_in])
        y.append(data[i+n_steps_in:i+n_steps_in+n_steps_out])
    return np.array(X), np.array(y)
n_steps_in = 18
n_steps_out = 18
X_train, y_train = create_dataset(train_scaled, n_steps_in, n_steps_out)
# 构建LSTM模型
model = Sequential()
model.add(LSTM(100, activation='relu', return_sequences=True, input_shape=(n_steps_in, 1)))
model.add(LSTM(100, activation='relu'))
model.add(Dense(n_steps_out))
model.compile(optimizer='adam', loss='mse')
# 训练模型
model.fit(X_train, y_train, epochs=200, batch_size=32, verbose=0)
# 预测第15周期
last_input = train_scaled[-n_steps_in:].reshape(1, n_steps_in, 1)
predicted_scaled = model.predict(last_input)
predicted = scaler.inverse_transform(predicted_scaled).flatten()
predicted = np.round(predicted).clip(min=1).astype(int)  # 后处理
# 可视化对比
time_labels = [f"{11+i//6}:{i%6 * 10:02d}" for i in range(18)]
```

```
    # 生成时间标签
plt.figure(figsize=(12,6))
plt.plot(time_labels, test_data, marker='o', label='实际值')
plt.plot(time_labels, predicted, marker='x', label='预测值')
plt.xticks(rotation=45)
plt.xlabel('时间')
plt.ylabel('数值')
plt.title('第15周期预测对比')
plt.legend()
plt.tight_layout()
plt.show()
# 预测第16周期（使用原始浮点预测值进行输入）
predicted_15_for_next = scaler.transform(predicted.reshape(-1, 1))  # 重新归一化处理后的整数
input_16 = predicted_15_for_next[-n_steps_in:].reshape(1, n_steps_in, 1)
predicted_16_scaled = model.predict(input_16)
predicted_16 = scaler.inverse_transform(predicted_16_scaled).flatten()
predicted_16 = np.round(predicted_16).clip(min=1).astype(int)  # 后处理
print("预测的第15周期数据: \n", predicted)
%预测结果部分可视化
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime, timedelta

# 读取Excel文件
file_path = '预测结果对比图.xlsx'
df = pd.read_excel(file_path, header=None)

# 手动定义列名
columns = ['实际包裹量', 'LSTM预测量', '分段预测', '有约束的分段预测']
df.columns = columns
```

```python

# 将数据转换为数值类型
for col in columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# 创建从11:00开始的时间序列（每10分钟一个单位）
start_time = datetime.strptime('10:50', '%H:%M')
time_series = [start_time + timedelta(minutes=10 * i) for i in range(len(df))]

# 将时间序列转换为字符串格式以便显示
time_labels = [t.strftime('%H:%M') for t in time_series]

# 创建图表
plt.figure(figsize=(14, 8))

# 绘制散点折线图
for col in columns[1:]:
    plt.plot(time_labels, df[col], marker='o', label=col, linestyle='-', linewidth=2)

# 绘制实际包裹量
plt.plot(time_labels, df[columns[0]], marker='o', label='实际包裹量', linestyle='-', linewidth=2, color='blue')

# 添加标签和标题
plt.xlabel('时间')
plt.ylabel('包裹量')
plt.title('预测结果对比图')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)  # 旋转X轴标签以便更好显示
plt.tight_layout()

# 显示图表
```

```python
plt.show()

%调度模型建立
import pandas as pd
import numpy as np

# ----------------------------
# 参数设置
# ----------------------------
VEHICLE_CAPACITY = 1000
own_vehicle_count = {
    '场地1': 29,
    '场地2': 14,
    '场地3': 36,
    '场地4': 31
}

# ----------------------------
# 读取数据
# ----------------------------
forecast = pd.read_excel("结果表1.xlsx")
forecast.columns = forecast.columns.str.strip()
print("□□ 表头：", forecast.columns.tolist())

# □ 修复线路编码格式问题
forecast['线路编码'] = forecast['线路编码'].astype(str)
forecast['线路编码'] = forecast['线路编码'].str.replace('－', '-', regex=False)
forecast['线路编码'] = forecast['线路编码'].str.replace('—', '-', regex=False)
forecast['线路编码'] = forecast['线路编码'].str.replace('﹣', '-', regex=False)
forecast['线路编码'] = forecast['线路编码'].str.replace(' ', '')  # 去空格

```

```python
# 自动提取节点与场地
forecast['节点'] = forecast['线路编码'].str.extract(r'-(\d{4})$')
forecast['场地'] = forecast['线路编码'].str.extract(r'(场地\d)')

# 获取预测字段
col_volume = [col for col in forecast.columns if '预测' in col and '货量' in col]
if not col_volume:
    raise ValueError("□ 未找到预测货量字段")
col_name = col_volume[0]

# ----------------------------
# 创建任务
# ----------------------------
def create_tasks(forecast_df, node_time):
    tasks = []
    forecast_node = forecast_df[forecast_df['节点'] == node_time]
    for _, row in forecast_node.iterrows():
        total_volume = row[col_name]
        line = row['线路编码']
        site = row['场地']
        trips = int(np.ceil(total_volume / VEHICLE_CAPACITY))
        for t in range(trips):
            volume = VEHICLE_CAPACITY if t < trips - 1 else total_volume % VEHICLE_CAPACITY
            if volume == 0:
                volume = VEHICLE_CAPACITY
            tasks.append({
                '线路': line,
                '货量': volume,
                '节点': node_time,
                '场地': site
```

```
            })
    return pd.DataFrame(tasks)

tasks_0600 = create_tasks(forecast, '0600')
tasks_1400 = create_tasks(forecast, '1400')

# ----------------------------
# 串点合并
# ----------------------------
def merge_tasks(tasks):
    if tasks.empty:
        return pd.DataFrame(columns=['线路', '货量', '节点', '场地'])
    tasks = tasks.sort_values('货量').reset_index(drop=True)
    merged = []
    while not tasks.empty:
        current = tasks.iloc[0]
        base_site = current['场地']
        group = [current]
        total_vol = current['货量']
        tasks = tasks.iloc[1:].reset_index(drop=True)
        drop_idxs = []
        for i, task in tasks.iterrows():
            if task['场地'] == base_site and len(group) < 3 and total_vol + task['货量'] <= VEHICLE_CAPACITY:
                group.append(task)
                total_vol += task['货量']
                drop_idxs.append(i)
        tasks = tasks.drop(drop_idxs).reset_index(drop=True)
        merged.append({
            '线路': '&'.join(g['线路'] for g in group),
            '货量': total_vol,
            '节点': current['节点'],
            '场地': current['场地']
        })
```

```python
    return pd.DataFrame(merged)

merged_0600 = merge_tasks(tasks_0600)
merged_1400 = merge_tasks(tasks_1400)
print("□ merged_0600 行数：", len(merged_0600))
print("□ merged_1400 行数：", len(merged_1400))

# ----------------------------
# 调度车辆
# ----------------------------
def schedule(tasks, own_count_dict):
    vehicle_usage = {site: 0 for site in own_count_dict}
    result = []
    for _, task in tasks.iterrows():
        site = task['场地']
        if vehicle_usage[site] < own_count_dict[site]:
            vehicle_usage[site] += 1
            vehicle = f"自有{vehicle_usage[site]}号"
        else:
            vehicle = "外部"
        result.append({
            '线路编码': task['线路'],
            '预计发运时间': task['节点'],
            '发运车辆': vehicle,
            '日期': '2024-12-16'
        })
    return pd.DataFrame(result)

result_0600 = schedule(merged_0600, own_vehicle_count)
result_1400 = schedule(merged_1400, own_vehicle_count)
print("□ result_0600 行数：", len(result_0600))
print("□ result_1400 行数：", len(result_1400))

# ----------------------------
# 合并输出
```

```
# ------------------------------
result_all = pd.concat([result_0600, result_1400], ignore_index=True)

if result_all.empty:
    print("□ 最终结果为空，请检查线路编码格式")
else:
    result_all.columns = result_all.columns.astype(str).str.strip()
    result_all = result_all[['线路编码', '日期', '预计发运时间', '发运车辆']]
    result_all.to_excel("结果表3.xlsx", index=False)
    print("□ 结果表3.xlsx 已成功生成 □")
%结果可视化分析
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import font_manager as fm
from pathlib import Path

# ---------- 配置中文字体（若环境中已安装宋体 / SimHei / 微软雅黑等自动生效） ----------
font_candidates = ['SimHei', 'Microsoft YaHei', 'PingFang SC', 'Noto Sans CJK SC']
for fc in font_candidates:
    if any(fc in f.name for f in fm.fontManager.ttflist):
        plt.rcParams['font.sans-serif'] = [fc]
        break
plt.rcParams['axes.unicode_minus'] = False  # 负号正常显示

# ---------- 读取结果表 3 ----------
file_path = "结果表3.xlsx"
df = pd.read_excel(file_path)

# ---------- 数据预处理 ----------
df['场地'] = df['线路编码'].str.extract(r'(场地\d)')
```

```python
df['车辆类型'] = df['发运车辆'].apply(lambda x: '外部' if str(x).strip() == '外部' else '自有')

out_dir = Path('.')  # 当前目录

# ---------- 1. 各场地自有 / 外部车辆使用次数 ----------
site_vehicle = df.groupby(['场地', '车辆类型']).size().unstack(fill_value=0)

ax1 = site_vehicle.plot(kind='bar')
ax1.set_title("各场地自有 / 外部车辆发运次数")
ax1.set_ylabel("次数")
ax1.set_xlabel("场地")
plt.tight_layout()
plt.savefig(out_dir / "各场地车辆使用次数.png", dpi=300)
plt.close()

# ---------- 2. 自有 vs 外部车辆占比 ----------
veh_type = df['车辆类型'].value_counts()

fig2, ax2 = plt.subplots()
veh_type.plot(kind='pie', autopct='%1.1f%%', startangle=90, ax=ax2)
ax2.set_title("自有 vs 外部车辆占比")
ax2.set_ylabel("")
plt.tight_layout()
plt.savefig(out_dir / "自有与外部车辆占比.png", dpi=300)
plt.close()

# ---------- 3. 不同发运节点需求数量 ----------
node_counts = df['预计发运时间'].value_counts().sort_index()

ax3 = node_counts.plot(kind='bar')
ax3.set_title("不同发运节点需求数量")
ax3.set_ylabel("次数")
```

```python
ax3.set_xlabel("发运节点 (0600 / 1400)")
plt.tight_layout()
plt.savefig(out_dir / "发运节点需求数量.png", dpi=300)
plt.close()

print("□ 已生成三张图片: ")
for f in ["各场地车辆使用次数.png", "自有与外部车辆占比.png", "发运节点需求数量.png"]:
    print("  -", f)
%引入容器后的调度模型
import pandas as pd
import numpy as np

# ----------------------------
# 参数定义
# ----------------------------
CAPACITY_NO_CONTAINER = 1000     # 不使用标准容器时容量
CAPACITY_CONTAINER = 800         # 使用标准容器时容量
LOAD_TIME_NO_CONTAINER = 45      # 装卸时间（分钟）
LOAD_TIME_CONTAINER = 10
MAX_STOPS_PER_TRIP = 3
OWN_VEHICLE_LIMIT = {
    '场地1': 29,
    '场地2': 14,
    '场地3': 36,
    '场地4': 31,
}
DATE = '2024-12-16'

# ----------------------------
# 读取并预处理数据
# ----------------------------
pred_total = pd.read_excel('结果表1.xlsx')
pred_total.columns = pred_total.columns.str.strip()

```

```python
# 修复线路编码格式
pred_total['线路编码'] = pred_total['线路编码'].astype(str)
pred_total['线路编码'] = pred_total['线路编码'].str.replace('－', '-', regex=False)
pred_total['线路编码'] = pred_total['线路编码'].str.replace('—', '-', regex=False)
pred_total['线路编码'] = pred_total['线路编码'].str.replace('﹣', '-', regex=False)
pred_total['线路编码'] = pred_total['线路编码'].str.replace(' ', '')

# 提取字段
pred_total['场地'] = pred_total['线路编码'].str.extract(r'(场地\d)')
pred_total['站点'] = pred_total['线路编码'].str.extract(r'站点(\d+)')
pred_total['节点'] = pred_total['线路编码'].str.extract(r'-(\d{4})$')

# ------------------------------
# 调度函数（默认自有车优先使用容器，除非任务大）
# ------------------------------
def dispatch_by_site_node(site, node, group):
    tasks = []
    for _, row in group.iterrows():
        remain = row['预测总货量']
        while remain > 0:
            chunk = min(remain, CAPACITY_NO_CONTAINER)
            tasks.append((row['站点'], int(chunk)))
            remain -= chunk

    tasks.sort(key=lambda x: x[1])
    schedule = []
    used_vehicle = 0

```

```
    while tasks:
        if used_vehicle < OWN_VEHICLE_LIMIT.get(site, 0):
            route, load = [], 0

            # 默认启用容器，除非任务过大
            preview_total = sum([v for _, v in tasks[:MAX_STOPS_PER_TRIP]])
            use_container = True if preview_total <= 900 else False
            cap = CAPACITY_CONTAINER if use_container else CAPACITY_NO_CONTAINER

            i = 0
            while i < len(tasks) and len(route) < MAX_STOPS_PER_TRIP:
                station, vol = tasks[i]
                if load + vol <= cap:
                    route.append(f"{site}-站点{station}-{node}")
                    load += vol
                    tasks.pop(i)
                else:
                    i += 1
                if load >= cap * 0.95:
                    break

            used_vehicle += 1
            schedule.append(["&".join(route), DATE, node, f"自有{used_vehicle}号", '是' if use_container else '否'])
        else:
            station, vol = tasks.pop(0)
            schedule.append([f"{site}-站点{station}-{node}", DATE, node, '外部', '否'])

    return schedule
```

```python

# ----------------------------
# 主调度过程
# ----------------------------
schedules = []
for (site, node), group in pred_total.groupby(['场地', '节点']):
    part = dispatch_by_site_node(site, node, group)
    schedules.extend(part)

# ----------------------------
# 保存结果表4
# ----------------------------
result_df = pd.DataFrame(schedules, columns=['线路编码', '日期', '预计发运时间', '发运车辆', '是否使用标准容器'])
result_df.to_excel('结果表4.xlsx', index=False)
print("□ 结果表4.xlsx 生成成功（容器使用优先策略已启用）")
%结果可视化分析
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from collections import defaultdict

# 设置中文字体（解决乱码问题）
matplotlib.rcParams['font.family'] = 'SimHei'  # 黑体
matplotlib.rcParams['axes.unicode_minus'] = False

# 读取结果表4
df_q3 = pd.read_excel("结果表4.xlsx")
df_q3.columns = df_q3.columns.str.strip()
df_q3['是否使用标准容器'] = df_q3['是否使用标准容器'].fillna('否')
df_q3['时间节点'] = df_q3['预计发运时间'].astype(str)

# 添加站点数统计列（修复图4报错）
```

```python
def count_stations(route_str):
    return route_str.count('站点')
df_q3['站点数'] = df_q3['线路编码'].apply(count_stations)

# 图1：饼图 - 容器使用比例（自有车辆）
own_q3 = df_q3[df_q3['发运车辆'].str.contains('自有')]
labels = ['使用容器', '未使用容器']
values = [len(own_q3[own_q3['是否使用标准容器'] == '是']),
          len(own_q3[own_q3['是否使用标准容器'] == '否'])]
plt.figure(figsize=(6, 6))
plt.pie(values, labels=labels, autopct='%1.1f%%', startangle=140)
plt.title('自有车辆中容器使用比例')
plt.axis('equal')
plt.tight_layout()
plt.savefig('图1_容器使用比例_饼图.png')
plt.show()

# 图2：柱状图 - 各类车辆使用数
external_q3 = df_q3[df_q3['发运车辆'] == '外部']
plt.figure(figsize=(6, 4))
plt.bar(['容器车次', '非容器车次', '外部车辆'], [values[0], values[1], len(external_q3)])
plt.ylabel('车次数')
plt.title('问题三各类车辆使用情况')
plt.tight_layout()
plt.savefig('图2_各类车辆使用数_柱状图.png')
plt.show()

# 图3：不同时间节点容器使用分布（自有车辆）
time_use = own_q3.groupby(['时间节点', '是否使用标准容器']).size().unstack(fill_value=0)
time_use.plot(kind='bar', stacked=True, figsize=(6, 4))
plt.title('不同时间节点容器使用分布（自有车辆）')
plt.xlabel('时间节点')
```

```
plt.ylabel('车次数')
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig('图3_不同时间节点容器使用_堆叠柱状图.png')
plt.show()

# 图4：容器 vs 非容器 平均串点站点数
grouped_station = own_q3.groupby('是否使用标准容器')['站点数'].mean()
grouped_station.plot(kind='bar', figsize=(5, 4))
plt.title('平均串点站点数（容器 vs 非容器）')
plt.ylabel('平均站点数')
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig('图4_平均串点站点数_柱状图.png')
plt.show()

# 图5：自有车辆容器/非容器任务分布
vehicle_task_counts = defaultdict(lambda: {'容器': 0, '非容器': 0})
for _, row in own_q3.iterrows():
    container = '容器' if row['是否使用标准容器'] == '是' else '非容器'
    vehicle_task_counts[row['发运车辆']][container] += 1
task_df = pd.DataFrame(vehicle_task_counts).T
task_df.plot(kind='bar', stacked=True, figsize=(10, 4))
plt.title('各自有车辆容器 vs 非容器任务数')
plt.ylabel('任务次数')
plt.xlabel('自有车辆编号')
plt.xticks(rotation=90)
plt.tight_layout()
plt.savefig('图5_自有车辆容器使用分布_堆叠柱状图.png')
plt.show()
%第四问
import pandas as pd
```

```python
import numpy as np
from collections import defaultdict

# 配置参数
capacity = 1000  # 单车容量
fleet_limit = {'场地1': 29, '场地2': 14, '场地3': 36, '场地4': 31}  # 自有车辆数
cost_self = 200
cost_external = 500

# 读取预测数据和原始调度结果
pred_path = "结果表1.xlsx"
result_q3_path = "结果表3.xlsx"

# 读取预测表
df_pred = pd.read_excel(pred_path)
df_pred.columns = df_pred.columns.str.strip()
df_pred = df_pred[df_pred['预测总货量'].notnull()].copy()
df_pred['线路编码'] = df_pred['线路编码'].astype(str)

# 提取场地与节点
df_pred['场地'] = df_pred['线路编码'].str.extract(r'(场地\d+)')
df_pred['节点'] = df_pred['线路编码'].str.extract(r'站点\d+ - (\d{4})')
df_pred = df_pred[df_pred['场地'].notnull() & df_pred['节点'].notnull()]

# 加入扰动 ±20%，最小值为1
np.random.seed(42)
df_pred['扰动后货量'] = df_pred['预测总货量'] * (1 + np.random.uniform(-0.2, 0.2, size=len(df_pred)))
df_pred['扰动后货量'] = df_pred['扰动后货量'].round().clip(lower=1).astype(int)

```

```python
# 调度模拟逻辑（贪心合并）
trips = []
for (site, node), group in df_pred.groupby(['场地', '节点']):
    volume_list = group['扰动后货量'].tolist()
    volume_list.sort(reverse=True)
    used = [False] * len(volume_list)
    for i, vi in enumerate(volume_list):
        if used[i]: continue
        load = vi
        used[i] = True
        for j in range(i+1, len(volume_list)):
            if not used[j] and load + volume_list[j] <= capacity:
                load += volume_list[j]
                used[j] = True
        trips.append((site, node, load))

# 分配车辆，自有优先
used_self = defaultdict(int)
trip_detail = []
for site, node, load in trips:
    if site not in fleet_limit:
        continue
    key = (site, node)
    if used_self[key] < fleet_limit[site]:
        vehicle_type = '自有'
        used_self[key] += 1
    else:
        vehicle_type = '外部'
    full = load >= capacity
    trip_detail.append((site, node, load, vehicle_type, full))

# 扰动后指标统计
df_sim = pd.DataFrame(trip_detail, columns=['场地', '节点', '货量', '发运车辆', '是否满载'])
```

```python
self_count = df_sim[df_sim['发运车辆'] == '自有'].shape[0]
ext_count = df_sim[df_sim['发运车辆'] == '外部'].shape[0]
full_ratio = df_sim['是否满载'].mean()
total_cost = self_count * cost_self + ext_count * cost_external

# 原始调度统计
df_q3_orig = pd.read_excel(result_q3_path)
df_q3_orig.columns = df_q3_orig.columns.str.strip()
orig_self = df_q3_orig[df_q3_orig['发运车辆'].str.contains('自有')].shape[0]
orig_ext = df_q3_orig[df_q3_orig['发运车辆'] == '外部'].shape[0]
orig_cost = orig_self * cost_self + orig_ext * cost_external

# 输出对比结果
print("问题四扰动仿真结果对比：")
print("----------------------------")
print(f"自有车辆使用次数：扰动前 = {orig_self}，扰动后 = {self_count}，变化 = {self_count - orig_self}")
print(f"外部车辆使用次数：扰动前 = {orig_ext}，扰动后 = {ext_count}，变化 = {ext_count - orig_ext}")
print(f"容器满载车次比例：扰动前 = 0.0%，扰动后 = {round(full_ratio * 100, 1)}%")
print(f"运输总成本：扰动前 = ¥{orig_cost}，扰动后 = ¥{total_cost}，节省 = ¥{orig_cost - total_cost}")
```

详细图表和代码结果请看数据包